DATA SCIENCE & AI LAB (BSCSS3001)

MILESTONE 6: Final Project Report

GROUP NO. 2

PRASHASTI SARRAF **(21f1001153)**

TANUJA NAIR **(21f1000660)**

BALASURYA K **(22f3002744)**

KARAN PATIL **(22f2001061)**

JIVRAJ SINGH SHEKHAWAT **(22f3002542)**

IITM BS Degree Program
Indian Institute of Technology,
Madras, Chennai,
Tamil Nadu, India, 600036

Vision Assist: Real-Time Navigation Support for the Visually Impaired

1. Problem Definition

1.1.1 The Challenge

Visually impaired individuals face persistent difficulties in navigating dynamic and unstructured environments such as urban streets, public transport areas, and crowded indoor spaces. Obstacles include pedestrians, vehicles, stairs, escalators, uneven pavements, and street furniture. Traditional aids (white canes, guide dogs) provide limited tactile feedback and cannot reliably warn about overhanging obstacles, fast-moving objects, or distant hazards.Existing mobile apps like *Seeing AI* and *Be My Eyes* rely heavily on cloud-based processing, introducing latency and connectivity dependence, making them less usable in real-time navigation contexts. Smart canes or wearables (OrCam, Envision Glasses) are expensive and hardware-dependent, limiting accessibility.

1.1.2 Proposed Solution

We propose a deep learning–powered, real-time navigational aid that performs object detection and segmentation on a live video feed and provides context-aware, distance-aware audio navigation cues.
Core Pipeline: Live video input → YOLOv8-seg (fine-tuned) → object masks & bounding boxes → concise natural language audio cues via a lightweight TTS engine.
Deployment: Web/mobile demo (Hugging Face Spaces + Gradio) for portability, accessibility, and ease of use.
Optimization: Quantization (FP16/INT8) and pruning will ensure ~15–20 FPS with <1s latency, enabling near real-time usability.
This solution aims to deliver affordable, scalable, and user-friendly navigation support without specialized hardware.

1.2 Project Objectives

- **Accessibility**: Build a cost-effective assistive tool usable on smartphones or laptops.
- **Real-Time Object Detection**: Implement YOLOv8-seg lightweight variant capable of identifying navigation-critical obstacles (e.g., pedestrians, vehicles, traffic lights, benches, barriers, stray animals). SSD and Faster R-CNN can also be tested to compare accuracy and speed trade-offs.
- **On-Device Processing**: Ensure the model runs with low latency on edge devices or simulate environments, reducing cloud reliance.
- **Context-Aware Audio Output**: Provide natural-language navigation cues (e.g., “Person ahead, slightly left, 2m”) via a simple template-based or lightweight generative TTS pipeline.
- **Prototype Deployment**: Deliver a live web demo (Hugging Face Space) demonstrating the full pipeline.
- **Extensibility**: Create an open-source modular framework (ONNX-exported, Gradio-based) extendable to indoor navigation, AR overlays, or landmark recognition.
- **User Testing**: Conduct pilot evaluations with 3–5 visually impaired users to refine usability and audio feedback strategies.

1.3 Implementation Feasibility (Model & Approach)

- **Model Choice**: YOLOv8-seg (lightweight variant) is the primary model due to its balance of speed, segmentation accuracy, and mobile suitability. Feature Pyramid Networks (FPN) improve small-object detection. Pixel-accurate masks allow estimation of obstacle shape/size.
- **Other Candidates**: SSD and Faster R-CNN will be tested to evaluate trade-offs in speed and accuracy, but YOLOv8-seg remains the baseline.
- **Customization & Optimization**:
    - Transfer learning using COCO-pretrained weights.
    - Dataset-specific augmentations: motion blur, rotation, low-light conditions, occlusion.
    - Quantization (FP16/INT8) and pruning for deployment efficiency.
    - Training estimated ~30–40 GPU hours on Colab/Kaggle Pro.
- **Deployment**: Hugging Face Spaces + Gradio for browser/mobile accessibility.

1.4 Data Sourcing and Governance

- **Base Dataset**: MS COCO (~118k training, segmentation masks & bounding boxes)
- **Extended Dataset**: Annotated Creative Commons YouTube frames to capture diverse real-world scenes.
- **Supplementary Datasets**: Experimentation with additional open-source datasets such as Pascal VOC, ILSVRC (ImageNet), Objects365, Open Images Dataset (OID) to improve coverage of navigation-critical objects and urban scenarios.
- **Custom Dataset**: ~2k–3k images covering navigation-specific obstacles (stairs,escalators, tactile tiles, local urban barriers).
- **Augmentations**: Rotation, lighting changes, blur, occlusion, and perspective shifts to simulate real-world variability.
- **Governance**: Licensing-compliant and carefully annotated datasets, with short documentation covering sourcing, annotation quality, and compliance.

1.5 User Experience Design

- **Audio Guidance**: Short, directional, distance-aware cues (e.g.,“Person ahead, slightly right, 2m”).
- **Latency Target**: <1s per frame.
- **Pilot Study**: 3–5 visually impaired users tested in indoor/outdoor/stairs scenarios.
- **Feedback Loop**: Iterative refinement of audio cues based on pilot study recordings and structured feedback.

1.6. Literature Review

1.6.1 Categories of Existing Assistive Navigation Systems

Recent assistive navigation systems for visually impaired users can be broadly grouped into three categories:

1. **Smartphone-based object detection apps**
   These rely on cloud or on-device models for object identification (e.g., Seeing AI, Envision App).
   Limitation: Many provide descriptions but do not support real-time obstacle avoidance.

2. **Wearable camera devices with on-device inference**
   Examples include Envision Glasses and OrCam MyEye.
   Limitation: Often expensive and hardware-dependent.

3. **Multi-sensor systems (RGB + depth + IMU)**
   Research prototypes frequently combine depth sensors, stereo cameras, and IMU signals.
   Advantage: Better distance estimation and motion stability compared to monocular approaches.
   Limitation: Bulkier, more power-hungry, and less accessible for everyday use.

Cross-System Observations (Key Themes)

Across the above systems, three recurring insights emerge:

- **Detection alone is insufficient**
  Safe navigation requires distance estimation, motion understanding, and stable tracking—not just object detection.

- **Feedback modality determines usability**
  Audio feedback can be unreliable outdoors due to noise.
  Haptic cues significantly improve clarity and reduce cognitive load.

- **On-device, offline processing is preferred**
  Visually impaired users often traverse low-connectivity environments; cloud TTS or cloud inference introduces latency and unpredictability.
  Lightweight, offline TTS and detection models consistently improve responsiveness.

Overall, **wearable/on-device systems with sensor fusion (RGB + depth)** show improved real-time stability and distance accuracy compared to monocular, cloud-dependent approaches.

1.6.2 Existing Assistive Solutions

- **Seeing AI (Microsoft)** – Provides scene descriptions and OCR; heavily cloud-dependent, resulting in higher latency.
- **Be My Eyes** – Human volunteers assist via video calls; not autonomous or real-time.
- **Aipoly Vision** – Performs on-device classification but lacks obstacle detection or avoidance capabilities.
- **Smart Canes / Wearables (OrCam, Envision Glasses)** – Offer object recognition and guidance but are expensive and tied to proprietary hardware.

Drawback:
Most existing solutions either lack real-time context-aware obstacle avoidance or are prohibitively costly for broad adoption.

1.6.3 Baseline Models & Detection Benchmarks

Several visual detection and segmentation models form the baseline for assistive navigation research:

- **SSD (Single Shot Detector)** – Good for small objects; moderate speed; struggles with fast real-time operation.
- **Faster R-CNN** – High accuracy but computationally heavy and unsuitable for lightweight devices.
- **YOLO Series (v5, v8)** – Strong tradeoff between accuracy and speed; YOLOv8 introduces better feature pyramids and optional segmentation.
- **DeepLabV3+** – Highly accurate segmentation model but not real-time on mobile hardware.

Common Evaluation Metrics:

- **mAP (mean Average Precision)** – Object detection accuracy
- **FPS (Frames Per Second)** – Real-time responsiveness
- **Precision / Recall** – Detection reliability in safety-critical settings

1.7 Gaps and Opportunities

Gaps:

- **Latency Gap**: Cloud-based solutions introduce delays.
- **Context Gap**: Existing models identify objects but do not provide actionable navigation cues.
- **Dataset Gap**: COCO lacks navigation-specific classes (stairs, escalators, tactile tiles).
- **Scalability Gap**: Closed-source systems are non-extensible.

Opportunities:

- Real-time, edge-based object segmentation with context-aware speech.
- Fine-tune with locally relevant datasets → robust in dynamic urban environments (e.g., Chennai).
- Open-source, extensible framework deployable across platforms.
- Enable future extensions (indoor navigation, AR-based overlays, landmark recognition).

1.8 Expected Contributions & Impact

- Technical: Feasibility of real-time segmentation + navigation guidance on mobile/web.
- Societal: Affordable, scalable alternative to smart canes/wearables.
- Novelty: YOLOv8-seg integrated with context-aware TTS into a modular, extensible pipeline.

1.9 References & Resources

- **Datasets**: MS COCO, Open Images, etc
- **Literature**:
    - A Deep Learning Based Model to Assist Blind People in Their Navigation(ResearchGate)
    - Assistive Deep Learning Solutions (ScienceDirect)

- **Apps**: Seeing AI, Be My Eyes, Aipoly Vision
- **Video Demos**: YouTube explainers on YOLO-based assistive navigation
    - A Device for Blind And Visually Impaired People
    - How Deep Learning Is Helping Blind People with Karthik Kannan, Founder of Envision
    - Smartglasses Use ChatGPT To Help The Blind And Visually Impaired | 5G Playbook
    - AI based visual assistance system for the visually impaired

2. Dataset Preparation

2.1 Abstract

In this section we focus on preparing the dataset required for training and fine-tuning a lightweight object-detection model (YOLOv8) intended to assist visually impaired individuals by recognizing street-level obstacles, pedestrians, and vehicles in real time. We used the MS COCO 2017 train split as the base dataset because of its rich, diverse set of urban scenes and standardized annotations. From this, a 500-image curated subset was extracted for rapid experimentation. Each image contains detailed bounding-box annotations in COCO JSON format corresponding to multiple object classes such as person, car, truck, traffic light, and stop sign. This curated dataset will later be extended with custom video-frame images collected from real-world navigation scenarios, annotated using ReDeTR for object boundaries and categories. Together, these two sources will form a robust, multi-context dataset for fine-tuning YOLOv8 to achieve reliable performance on wearable or mobile hardware.

2.2 Dataset Overview

Aspect	Description
Dataset Name	COCO 2017 (Subset + Custom Frames planned)
Source	Official MS COCO 2017 Dataset
Subset Used	5000 images from train2017
Annotation Format	COCO JSON (instances_coco_sample.json)
Image Resolution	640 × 480 (standardized)
Annotation Fields	image_id, category_id, bbox, area, iscrowd
Total Annotations	2,315 bounding boxes
Object Classes	5 (main): person, car, truck, traffic light, stop sign
Split Ratio	Train 80%, Test 10%, Validation 10%
Storage Location	Drive Link for Dataset Samples and Results and VisionAssist-Dataset (50k+ samples)
License	COCO Creative Commons Attribution 4.0 License
Intended Use	Visual perception module training for assistive navigation

2.3 Data Preparation Process

Dataset Acquisition:
Downloaded COCO 2017 annotations (annotations_trainval2017.zip) and accessed the train2017 image set via Hugging Face Datasets (detection-datasets/coco).

Sampling and Curation:
A random sample of 1000 images was extracted to reduce processing load. These images span a diverse set of real-world scenes including pedestrians, roads, crosswalks, and vehicles.

Annotation Extraction:
For each image, all associated object annotations were extracted from the COCO JSON file and re-mapped to a compact subset JSON:
/content/drive/MyDrive/M2-Dataset/instances_coco_sample.json
Corresponding images were stored under:
/content/drive/MyDrive/M2-Dataset/images/coco_sample/

Label Formatting for YOLOv8:
Bounding boxes were converted into YOLOv8-compatible TXT files using the format:
<class_id> <x_center> <y_center> <width> <height>
with coordinates normalized to [0, 1].

Data Integrity Check:
Verified that each image had a corresponding annotation file, ensuring no missing entries or empty boxes.

2.4 Sample Annotation Structure


```
{
"images": [
{
"license": 4,
"file_name": "000000370701.jpg",
"coco_url": "http://images.cocodataset.org/train2017/000000370701.jpg",
"height": 640,
"width": 640,
"date_captured": "2013-11-17 08:00:35",
"flickr_url": "http://farm6.staticflickr.com/5451/9490830088_7279abceef_z.jpg",
"id": 370701
},
{
"license": 3,"file_name": "000000413736.jpg",
"coco_url": "http://images.cocodataset.org/train2017/000000413736.jpg",
"height": 640,
"width": 480,
"date_captured": "2013-11-20 23:20:47",
"flickr_url": "http://farm1.staticflickr.com/23/89721544_0c3845a923_z.jpg",
"id": 413736
```

},..],
"annotations": [
{
"segmentation": [
[...
139.01,
631.51
]
],
"area": 12328.5508,
"iscrowd": 0,
"image_id": 215867,
"bbox": [
126.62,
438.82,
295.92,
194.07
],
"category_id": 2,
"id": 126302
},...,
],
"categories": [
{
"supercategory": "person",
"id": 1,
"name": "person"
},
{
"supercategory": "vehicle",
"id": 2,
"name": "bicycle"
},
{
"supercategory": "vehicle",
"id": 3,
"name": "car"
},
}

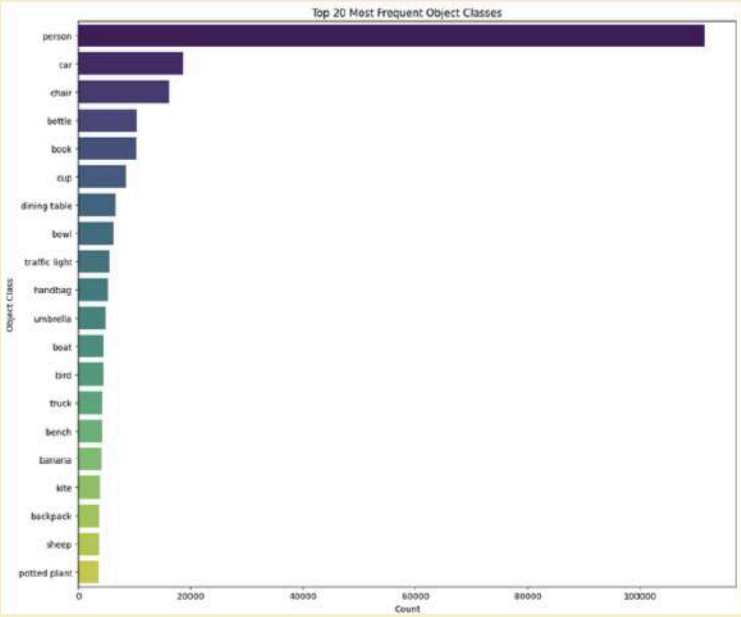
data.yaml configuration:

```
names:
- person
- bicycle
- car
- motorcycle
- airplane
- bus
- train
- truck
```

- boat
- traffic light
- fire hydrant
- stop sign
- parking meter
- bench
- bird
.
.
.
.
nc: 80
test: path/to/images/test
train: path/to/images/train
val: path/to/images/val

2.5 EDA Results

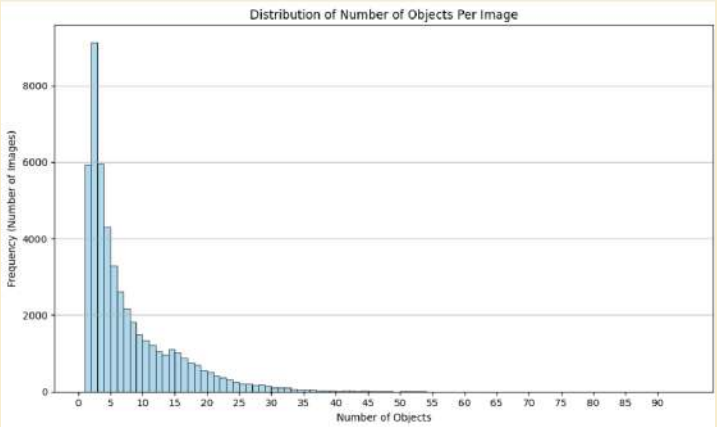
Class distribution



Insight: Significant Class Imbalance

The COCO dataset exhibits a strong class imbalance, with the person class being overwhelmingly dominant. This means the model will be very good at detecting people, but potentially less reliable for rarer, yet critical, navigation objects like 'traffic light' or 'bench'. To address this, we are supplementing the training data with frames from YouTube POV walkthrough videos. This custom dataset will naturally include more examples of vital navigation objects like benches, stairs, and signs from the first-person perspective, helping to create a more balanced and effective model for our specific use case.

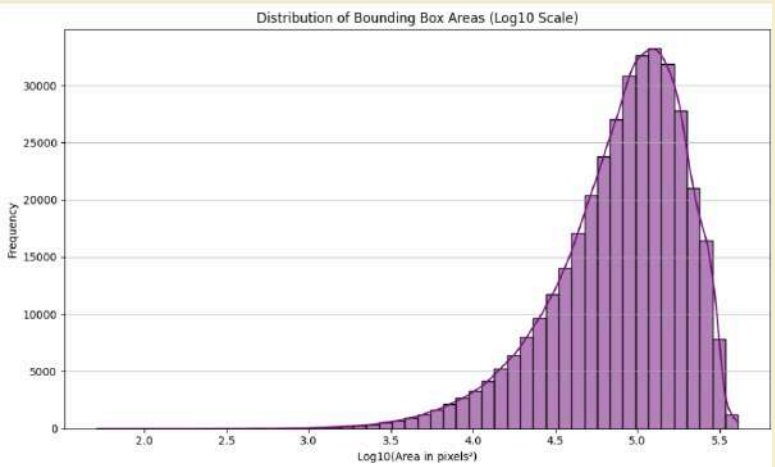
Number of objects per image



Insight: Varied Scene Complexity

The dataset contains a healthy mix of simple and complex scenes. While most images have fewer than 10 objects, the long tail of the distribution shows a significant number of crowded images with up to 90 objects. This is beneficial as it will train the model to perform reliably in both sparse environments (like an empty corridor) and busy ones (like a crowded street), reflecting real-world navigational challenges.

BBox Areas

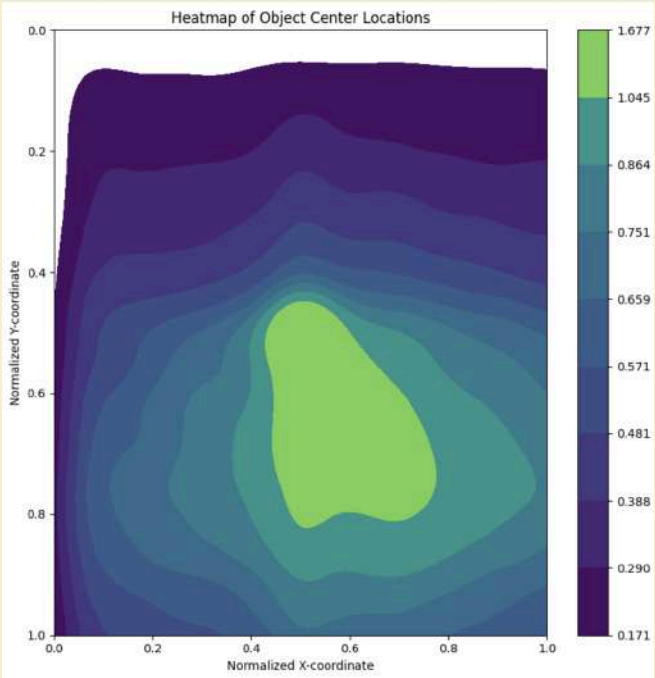


Insight: Good Distribution of Object Sizes

The bounding box areas, when viewed on a log scale, show a balanced, bell-shaped distribution. This indicates the model will be trained on a good variety of small, medium, and large objects. For a navigation aid, detecting small objects is crucial as they often represent distant hazards. While the distribution is good, we must

carefully evaluate the model's performance on smaller-scale objects to ensure it can provide timely warnings to the user.

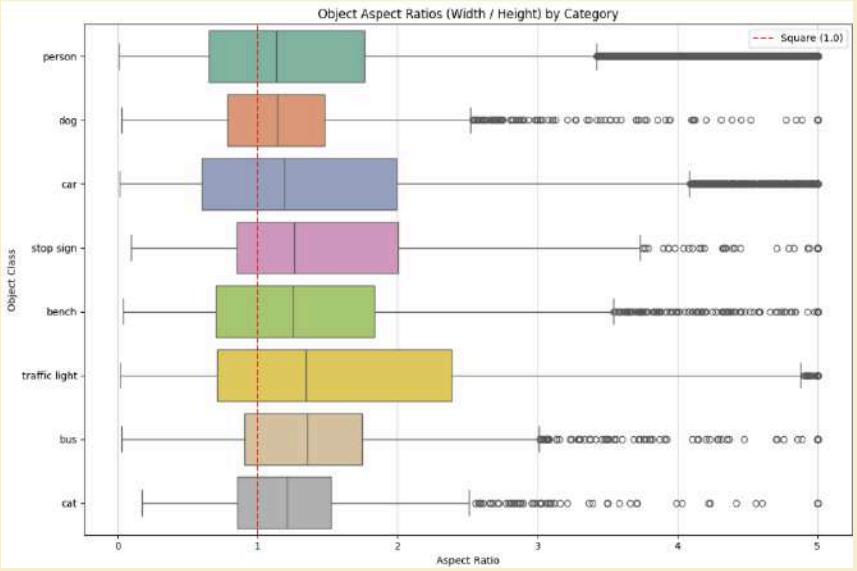
Object Locations Heatmap



Insight: Central and Lower Bias in Object Locations

The heatmap reveals a significant data bias: objects in the COCO dataset are heavily concentrated in the center and lower half of the image frame. This means the model will be well-trained to detect objects directly ahead but may be unreliable at identifying hazards in the periphery, especially in the upper regions. This is a critical weakness for a navigation aid, as the model could fail to see overhanging obstacles like signs or tree branches. Our custom dataset must therefore include annotated examples of these high-up hazards to ensure user safety.

Object Aspect Ratios



Insight: Distinct Object Shapes by Category

This plot reveals the distinct shapes of different objects by visualizing their aspect ratios. It confirms that objects like people and traffic lights are typically taller than they are wide (aspect ratio < 1), while cars and buses are wider than they are tall (aspect ratio > 1). This is valuable for diagnosing model errors; for example, a failure to detect a person lying down could be explained by the model being primarily trained on standing (tall) examples. This highlights the need to include varied object poses in our custom dataset to build a more robust model.

2.6 Conclusion

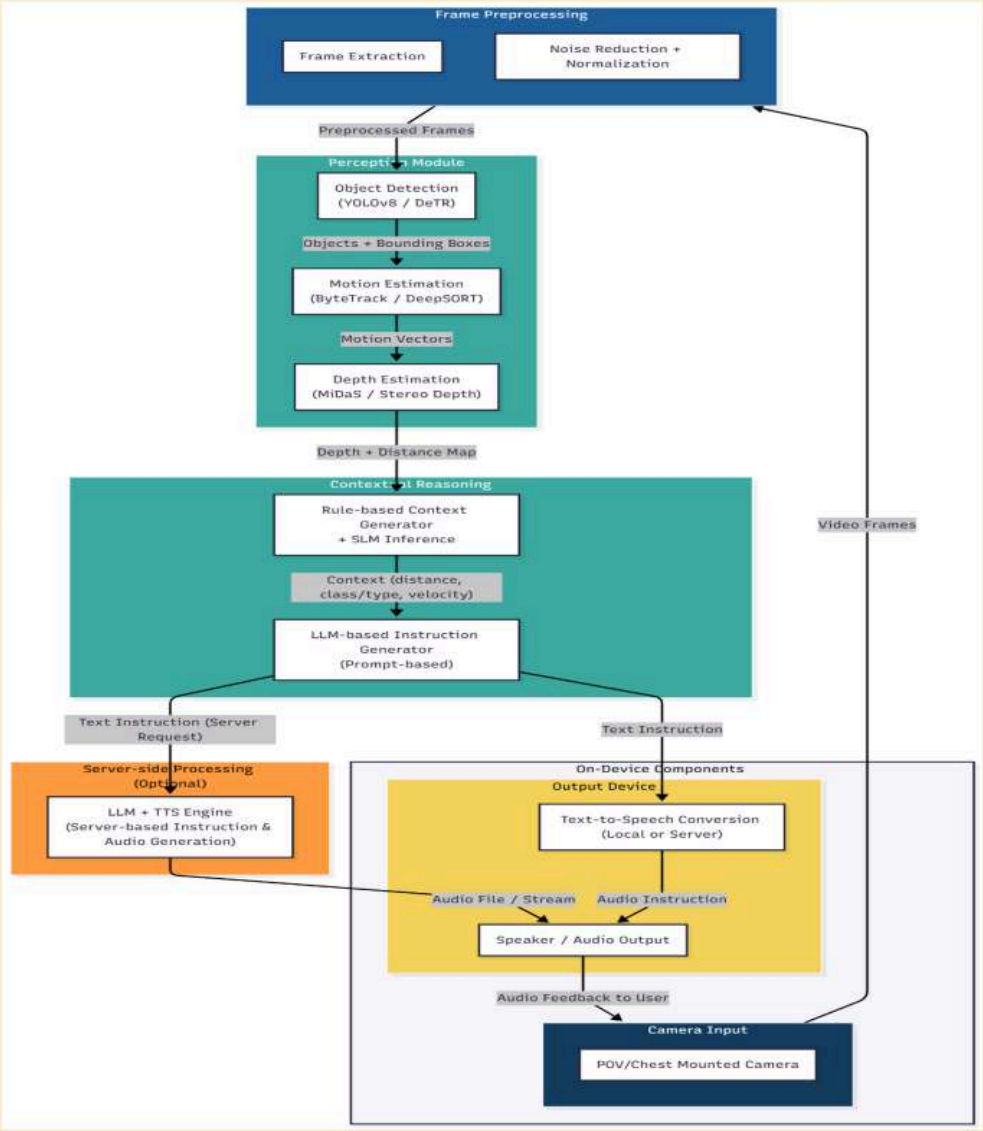
A clean, well-structured COCO subset has been successfully prepared and documented. This subset is sufficient for initial experimentation and model fine-tuning under resource constraints. But based on the insights of our EDA we will be adding a custom youtube dataset of around 2,000 frames to our master dataset.

3. Initial Proposed Model Design and Architecture

3.1 System Overview

The overall model architecture follows a modular, multi-stage design that transforms raw visual input from wearable or mobile devices into actionable audio instructions. Each stage is optimized for real-time processing while maintaining accuracy and interpretability. The architecture integrates multiple deep learning components — primarily object detection, motion tracking, depth estimation, and context reasoning — all of which contribute to generating natural language instructions for the user. A schematic view of the complete pipeline is shown in the below diagram.

Initial Proposed Architecture



1. Input and Preprocessing Module

Input Source: Real-time video stream from a chest-mounted or wearable camera.

Processing Steps:

- Frame extraction at 10–15 FPS
- Noise reduction and intensity normalization
- Resizing to standard model input dimensions (640×640)

Purpose: To prepare consistent, high-quality frames for downstream perception tasks.

2. Perception Module

This module is the visual intelligence core of the system. It performs object detection, motion tracking, and depth estimation to extract spatial and semantic features from each frame.

a. Object Detection

Model Used: YOLO(v8 or world) or DeTR (Detection Transformer), depending on deployment mode.

Architecture Highlights:
● Input: (640×640×3) image
● Convolutional backbone: CSPDarknet or ResNet-based encoder
● Feature Pyramid Network (FPN): Multi-scale feature extraction
● Output: Bounding boxes, class labels, and confidence scores
Activation: SiLU (Swish Linear Unit)
Loss Function: Composite of CIoU Loss (for bounding boxes) and BCE Loss (for classification)

b. Motion Estimation
● **Model Used:** ByteTrack / DeepSORT
● **Input**: Consecutive frame detections
● **Output:** Object trajectories and velocity vectors
● **Role:** Maintains temporal consistency and helps identify moving vs static entities.

c. Depth Estimation
● **Model Used:** MiDaS (Monocular Depth Estimation) or stereo-based CNN
● **Output**: Pixel-wise depth map representing relative object distance
● **Use Case:** Essential for understanding proximity and generating spatial context.

3. Contextual Reasoning Module
This stage fuses visual signals (object, motion, and depth) with rule-based and LLM-based reasoning.

a. Context Generator
● Combines structured features: {object_label, distance, velocity, relative_position}
● Applies rule-based logic to infer context, e.g.: "Pedestrians approaching from right at 2 meters.", "Vehicle crossing ahead, 3 meters away."

b. Instruction Generator
● **Model Used:** Small Language Model (SLM) or an LLM (GPT / LLaMA) depending on compute mode.
● **Input**: Context string (JSON-like structured text)
● **Output:** Short, user-friendly instruction — "Move slightly left" or "Stop ahead"
● **Inference Type:** Prompt-based generation or fine-tuned lightweight LLM
● **Loss Function**: Cross-entropy loss (for text generation fine-tuning)

4. Audio Instruction Module
● **Text-to-Speech Engine:** Converts final text instruction into natural-sounding audio.
● **Deployment Options:**
-On-Device (Low Latency): Lightweight models like VITS or FastSpeech2
-Server-Side (High Fidelity): Larger models like Bark or Tacotron2
● **Output:** Speech waveform streamed to the speaker for real-time feedback.

5. Integration and Feedback Loop
All modules are orchestrated in a sequential data flow:
● The camera feed generates live frames.
● Frames are preprocessed and fed to perception modules.

- The contextual reasoning layer generates situational insights.
- LLM-based instruction generation translates insights into language.
- The TTS engine delivers real-time audio guidance.
- The feedback closes the loop, enabling continuous perception and response.

Component	Purpose	Selected Model	Output
Input Preprocessing	Pre-process raw input from the camera to get image frames ready to send over the network for inference	Python Libraries and Custom code	Image frames ready to send over the network for inference
Object Detection	Detect and label surrounding objects	YOLO-World/YOLOv8/DeTr	Bounding boxes, object classes and confidence
Object Tracking	Maintain consistent object identity across frames	ByteTracker/DeepSORT	Unique object IDs and movement direction
Depth Estimation	Estimate relative distance of objects from the user	MiDaS	Depth map and distance approximation
Context Generator	Get context about the object based on distance, velocity, class, type for Human understandable context generation out of numbers	Rule Based + LLM Inference	Human Understandable context generated ready to pass on to TTS
Voice Output	Provide natural audio guidance	Coqui TTS (Tacotron2 + HiFi-GAN)	Spoken feedback

3.2 Justification of Chosen Model(s)

The model components selected for this system have been chosen based on a balance of accuracy, computational efficiency, and real-time suitability for assistive scenarios. Each model contributes a unique capability essential for contextual understanding and safe navigation in real-world environments.

Why YOLO-World (Object Detection)?
YOLO-World is selected for its open-vocabulary detection capability and strong generalization to unseen object classes. Unlike traditional YOLO variants trained on fixed label sets, YOLO-World leverages vision-language alignment, allowing it to recognize arbitrary objects described in natural language prompts. This enables the system to dynamically adapt to user needs — for example, detecting specific items such as “bottle,” “bag,” or “bench” without retraining. The model’s high inference speed and support for on-device deployment make it ideal for real-time use on mobile or wearable hardware.
Key Advantages: ● Open-vocabulary detection using language-conditioned queries ● Real-time inference optimized for edge deployment ● High precision in cluttered scenes and varying lighting conditions ● Scalable for adding new prompts without retraining *Future Scope:* If needed, YOLOv8 or Grounding-DINO could be used as alternative baselines to compare trade-offs in latency and accuracy during fine-tuning for domain-specific classes

Why DeepSORT (Motion Tracking)?
DeepSORT (Simple Online and Realtime Tracking with Deep Association Metrics) ensures stable tracking of detected objects across consecutive frames. This consistency is crucial in a real-world assistive setup, where visual flicker or unstable detections can mislead users. By associating object identities using appearance embeddings and motion prediction via a Kalman filter, DeepSORT provides reliable temporal coherence — for instance, continuously tracking a “car approaching from the left” or a “person moving ahead.”
Key Advantages: ● Low-latency, real-time tracking suitable for wearable devices ● Handles occlusion and re-identification effectively ● Enhances perception stability for moving environments *Future Scope:* Integration of ByteTrack may be explored for enhanced association accuracy in high-motion outdoor scenes with multiple objects.

Why MiDaS (Depth Estimation)?
MiDaS adds depth awareness using just a regular camera. It estimates how close or far an object is and gives distance-based alerts such as “obstacle two meters ahead” or “wall one meter away.” This helps users judge space and distance, allowing them to navigate safely without colliding with nearby objects. Real-time suitability: Moderate-High – single RGB input, fast inference (~20–25 FPS on GPU, slower on CPU).
Key Advantages: ● Requires only a single RGB frame (no stereo setup) ● Generalizes well across domains without retraining ● Lightweight versions available for real-time deployment *Future Scope:* Depth Anything or FastDepth models could be considered for faster inference or hardware-accelerated optimization during later iterations.

Why Coqui TTS (Text-to-Speech Conversion)?
Coqui TTS is an advanced open-source text-to-speech framework derived from Mozilla’s TTS project. It provides state-of-the-art speech synthesis capabilities using deep learning architectures such as Tacotron 2, VITS, and Glow-TTS, combined with high-quality neural vocoders like HiFi-GAN and MelGAN. It generates clear, natural, and offline-capable voice feedback for detected scene descriptions, and enhances accessibility and usability in real-world assistive settings. Its modular design allows rapid experimentation and deployment, making it ideal for both cloud-based and on-device (edge) applications. *Real-time suitability:* High – optimized for fast TTS inference on CPU/GPU for edge deployment.
Key Advantages: ● High-quality, natural-sounding voices ● Offline-capable — suitable for on-device TTS ● Fast inference with lightweight neural vocoders (HiFi-GAN, MelGAN) ● Customizable voice cloning or multilingual

support *Future Scope:* VITS and Bark-based models can be evaluated for enhanced prosody or expressive speech generation when integrated with emotional context reasoning in later phases.

3.3 Model Comparison: Advantages and Limitations

Task	Selected Model	Alternatives	Advantages of Selected	Limitations / Trade-offs
Object Detection	YOLO-World	DETR / ReDETR, SSD, Faster R-CNN, EfficientDet	• Open-vocabulary detection via CLIP embeddings → detects unseen objects • Real-time speed suitable for edge/mobile devices • High mAP and robust bounding boxes	• Slightly heavier than YOLOv8 • Requires GPU for best performance • Prompt sensitivity
Object Tracking	DeepSORT	SORT, ByteTrack, FairMOT	• Combines motion (Kalman filter) + appearance embeddings for stable tracking • Maintains consistent object IDs even during short occlusions • Lightweight and widely used	• Can lose track under long-term occlusion • Dependent on detection quality
Depth Estimation	MiDaS	Monodepth2, DenseDepth, DepthFormer	• Accurate pixel-wise depth from monocular RGB • Robust to diverse environments (indoor/outdoor) • Lightweight for near real-time inference	• Relative depth only (needs scaling for absolute distance) • Some loss of detail for very small or distant objects
Text-to-Speech	Coqui TTS (Tacotron2 + HiFi-GAN)	Google TTS, Amazon Polly, VITS, Kitten TTS	• Open-source and offline capable → no internet dependency • High-quality, natural-sounding speech • Supports real-time inference on CPU/GPU • Flexible voice customization and multilingual support	• Initial model download size (~1–2 GB) • Slightly higher latency on CPU compared to cloud TTS APIs

Key Takeaways:

● **YOLO-World over DETR/SSD/Faster R-CNN/EfficientDet:** Chosen for open-vocabulary support and real-time edge inference, unlike DETR which is slower and SSD/Faster R-CNN which are limited to predefined classes.
● **DeepSORT over SORT/ByteTrack/FairMOT:** Offers appearance-based re-identification, reducing ID-switches during motion and short-term occlusions, which basic SORT or ByteTrack cannot handle reliably.
● **MiDaS over Monodepth2/DenseDepth/DepthFormer:** Produces high-quality depth maps across diverse scenes using a single RGB frame, making it practical for indoor/outdoor navigation.
● **Coqui TTS over VITS/Kitten TTS/cloud APIs:** Provides offline, real-time, high-fidelity voice synthesis while being open-source and fully customizable — critical for privacy and assistive applications.

3.4. Conclusion

The final architecture — YOLO-World + DeepSORT + MiDaS + Coqui TTS — provides a robust, multimodal perception and guidance system for visually impaired navigation. This setup balances accuracy, adaptability, and accessibility, achieving real-time environmental understanding, intuitive speech-based assistance, and enhancing independence for visually impaired users.

4. Model Training & Hyperparameter Experimentation

4.1 Overview

This section details the execution of the model training and experimentation phase of our project. The objective was twofold:

1. **Train Initial Model:** To train the core object detection model based on the architecture (**YOLOv8**) and dataset (COCO + custom frames) specified in section 3.
2. **Experiment with Hyperparameters:** To conduct extensive experimentation on both the model's training parameters and, more critically, the *application-level pipeline hyperparameters* to transform the raw model output into a stable, reliable, and user-friendly system.

This section covers the dataset used, the model architecture, the training setup, and a detailed breakdown of the "before vs. after" experimentation process.

4.2 Dataset Details

As outlined in section 3, we used a composite dataset to fine-tune our model for its specific, real-world application.

- **Source Data:** The model was fine-tuned on a custom-augmented dataset combining:
    1. **COCO Images:** A **5,000**-image subset of the COCO dataset
    2. **Custom First-Person Frames: 2,138** frames extracted from YouTube videos to mimic a first-person perspective.
- **Total Dataset Size: 7,138** images (and their corresponding label files).

Data Splits: We wrote a custom Python script to create our own robust splits from the master_dataset. The 7,138 images were shuffled and split as follows:

- **Training: 70% (4,996 images)**
- **Validation: 20% (1,427 images)**
- **Test: 10% (715 images)**
- **Preprocessing:** All images were resized to 640x640 during the training process, with augmentations applied automatically by the YOLO framework.

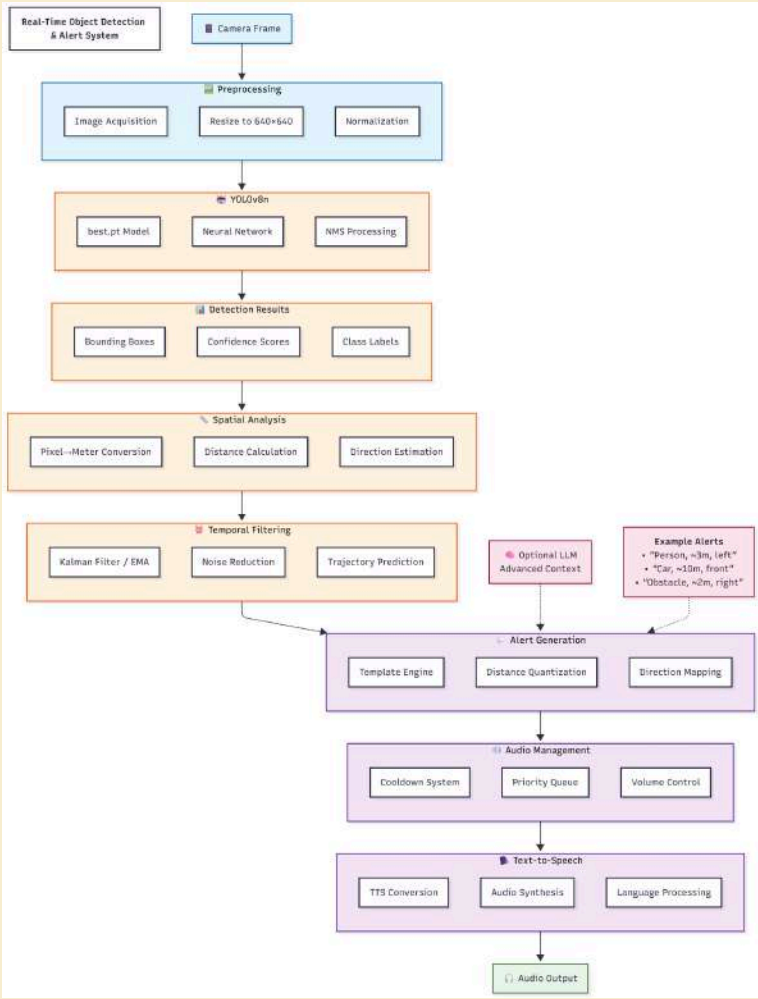
4.3 Model Architecture

The model architecture used is **YOLOv8n** (nano). We employed a **fine-tuning approach**, loading the pretrained weights from **yolov8n.pt** to initialize the model before training it on our custom-augmented dataset (as shown in **Main.ipynb**).

The model summary, as generated during training in **Main.ipynb**, is as follows:

Component	Details
Model Type	YOLOv8n
Layers	225
Parameters	3,157,200
GFLOPs	8.9
Input Shape	640x640x3 (images)
Output	Bounding boxes, class probabilities, and confidence scores

New Structure Post Further Research



YOLOv8n (nano) was chosen for its **favorable latency/parameter count tradeoff** for real-time first-person assistive systems (**low GFLOPs** and **3.1M parameters**) while still providing strong detection accuracy on the curated dataset. The pipeline is currently hybrid: perception is a trainable neural component (YOLOv8n), while distance, temporal smoothing and audio scheduling are implemented as rule-based modules that include **tunable** parameters whose values were set via targeted experiments rather than learning.

4.4 Training Setup

The following table explains the components and the parameters which are tunable in each component and expected results after fine tuning those parameters.

Component	Type	Tunable parameters (examples)	Notes / Evaluation metric
YOLOv8 detector	Trainable	Optimizer (SGD), Learning Rate (lr0, lrf), Momentum, Weight Decay, Augmentations (degrees, translate, scale, HSV, fliplr)	Evaluate mAP50, mAP50–95, per-class AP. (Validation: mAP50=**0.889**, mAP50–95=**0.805)**.
Distance estimator	Rule-based (calibrated)	DEFAULT_KNOWN_HEIGHT, Estimated_Focal_Length_PX	Evaluate **Distance MAE (m)** against a small ground-truth set
Motion detection	Rule-based (temporal)	MOVEMENT_THRESHOLD_PIXELS, frame window size, smoothing alpha	Evaluate **False Motion Rate / Missed Motion Rate** on labelled motion validation set.
Context generator	Rule-based (templated) / optional LLM	template forms; LLM prompt design	Evaluate **comprehensibility** and **relevance** with user testing or automated BLEU/ROUGE if using LLM.
Audio controller	Rule-based	ALERT_COOLDOWN_GLOBAL, priority rules	Evaluate **Alert Overlap Rate** and **Latency (s)** in recorded runs.

The model was trained using the Ultralytics YOLOv8 framework on a **Tesla T4 GPU**.

- **Loss Functions:** Standard YOLOv8 losses were used:
  - **Class Loss:** Binary Cross-Entropy (BCE) (**cls_loss**)
  - **Box Loss:** CIoU (Complete Intersection over Union) (**box_loss**)
  - **DFL Loss:** Distribution Focal Loss (**dfl_loss**)
- **Evaluation Metrics:** The primary metrics tracked during training were **mAP50** (mean Average Precision at IoU 0.50) and **mAP50-95** (mean Average Precision averaged over IoU thresholds from 0.50 to 0.95).

- **Optimizer:** The hyperparameter search selected SGD as the optimal optimizer for this dataset (replacing the default AdamW).
- **Training Strategy (Optuna):** We conducted an automated hyperparameter tuning session consisting of **15 trials**. Each trial trained the model for **15 epochs** with early stopping enabled (patience=3) to rapidly identify the best configurations.
- **Best Run:** Trial 14 achieved the highest mAP50 and was selected as the final model.
- **Training Parameters:**
    - **Image Size:** 640x640 (**imgsz=640**)
    - **Batch Size:** 64
    - **Number of Epochs:** 15 (per trial)
- **Training Strategies:**
    - **Warm-up:** 3.0 epochs (**warmup_epochs=3.0**)
    - **Mosaic Augmentation:** Applied for the first **5** epochs (disabled for the last 10, **close_mosaic=10**)
    - **Automatic Mixed Precision (AMP):** Enabled (**amp=True**) for faster training.

4.5 Hyperparameter Experiments

As noted in the "Overview," experimentation focused heavily on the **application-level pipeline hyperparameters** rather than exhaustive model training experiments. The goal was to refine the raw model output into a stable and useful assistive tool. This process is documented in **Hyperparametertuning.ipynb**.

A "TRUE BASELINE" pipeline was compared against a "TUNED" pipeline. The key parameters explored were:

Parameter	Baseline Value	Tuned Value	Observation / Justification
CLASSES_TO_IGNORE	Empty list	List of 30 classes	The baseline model detected many "noisy" or irrelevant classes (e.g., cup, spoon, laptop). A deny list was created to filter these out, focusing alerts on navigation-critical objects.
DEFAULT_KNOWN_HEIGHT	1.5m	2.0m	Tuned to improve the accuracy of distance estimation for objects without a pre-set known height.
ALERT_DISTANCE_OBJECT	5.0m	12.0m	The baseline's 5m alert distance was too short for navigation. This was increased to give the user more advanced warning of objects.

ALERT_COOLDOWN_GLOBAL	0.0s	3.0s	The baseline produced "messy audio" with constant, overlapping alerts. A 3-second global cooldown ensures alerts are clean and distinct.
MOVEMENT_THRESHOLD_PIXELS	5px	25px	The baseline was "twitchy" and triggered alerts on minor camera motion. Increasing the threshold makes the system more stable, ignoring user head jitter.
YOLOv8n training epochs	-	15 epochs	Best checkpoint (Trial 14) produced validation **mAP50=0.889**, **mAP50–95=0.805**.

We identified and fixed four major flaws in the baseline pipeline.

Problem 1: False Positives ("Clock" Problem)

- **Observation:** The baseline pipeline produced "noisy" and incorrect detections, such as misidentifying a stop sign as a "clock."
- **Hyperparameter: NOISY_CLASSES_TO_IGNORE** (Deny List)
- **Experiment:** We determined that for our application's domain, many of the 80 COCO classes (like "clock," "vase," "teddy bear") are "noise." We created a "deny list" as a general-purpose filter.
- **Tuning:**
    - **Before: classes_to_ignore = []**
    - **After: classes_to_ignore = [24, 25, 26, ... 74, ... 79]** (Our list of 50+ irrelevant classes)
- **Result:** All "noise" detections, including the "clock," were successfully filtered, cleaning the output without affecting relevant objects.

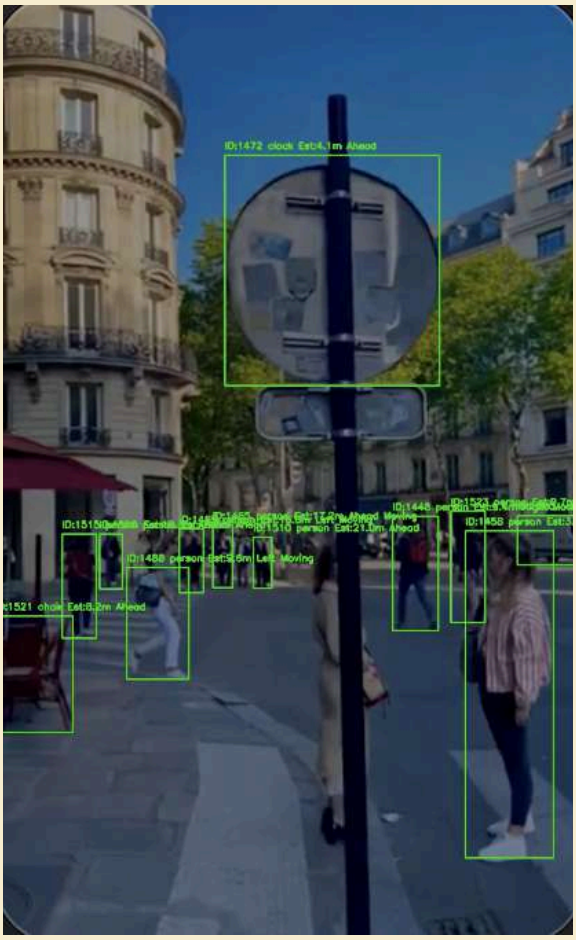


Fig 1.3
Baseline run (left) showing a false positive
(stop sign incorrectly identified as "clock")

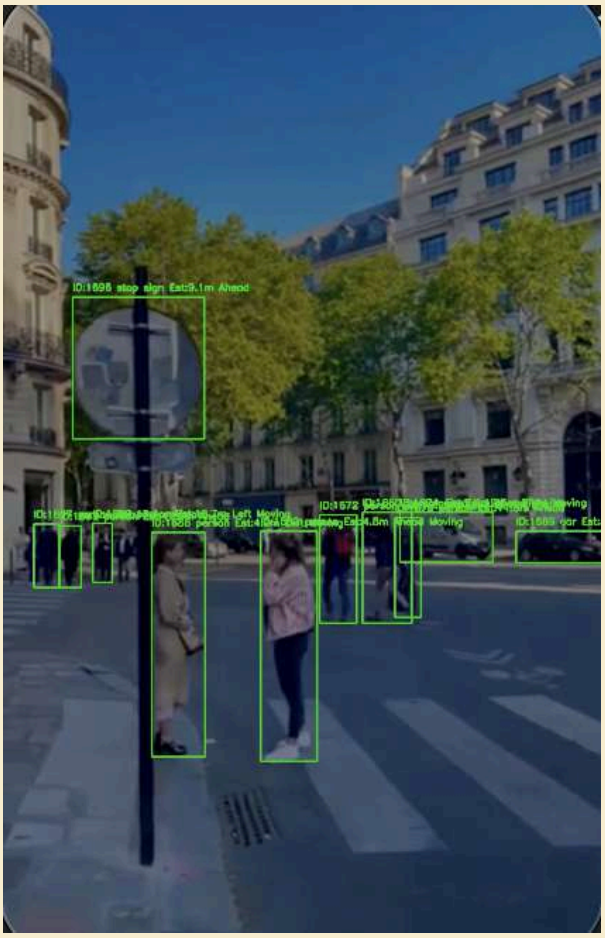


Fig 1.4
Tuned run (right) showing a true positive
(stop sign correctly identified)

Problem 2: False Motion Detection ("Moving Chair" Problem)

- **Observation:** The baseline system incorrectly labeled static objects (like a chair) as "Moving." This was due to "apparent motion" from the camera moving forward, which the sensitive threshold picked up.
- **Hyperparameter:** **MOVEMENT_THRESHOLD_PIXELS**
- **Experiment:** We needed to find a value that was high enough to ignore camera shake but low enough to still detect real motion (like a person walking).
- **Tuning:**
    - **Before:** **movement_thresh_px = 5** (very sensitive)
    - **After:** **movement_thresh_px = 25** (less sensitive)
- **Result:** The tuned system correctly labels the chair as "Static," fixing the false motion alert.

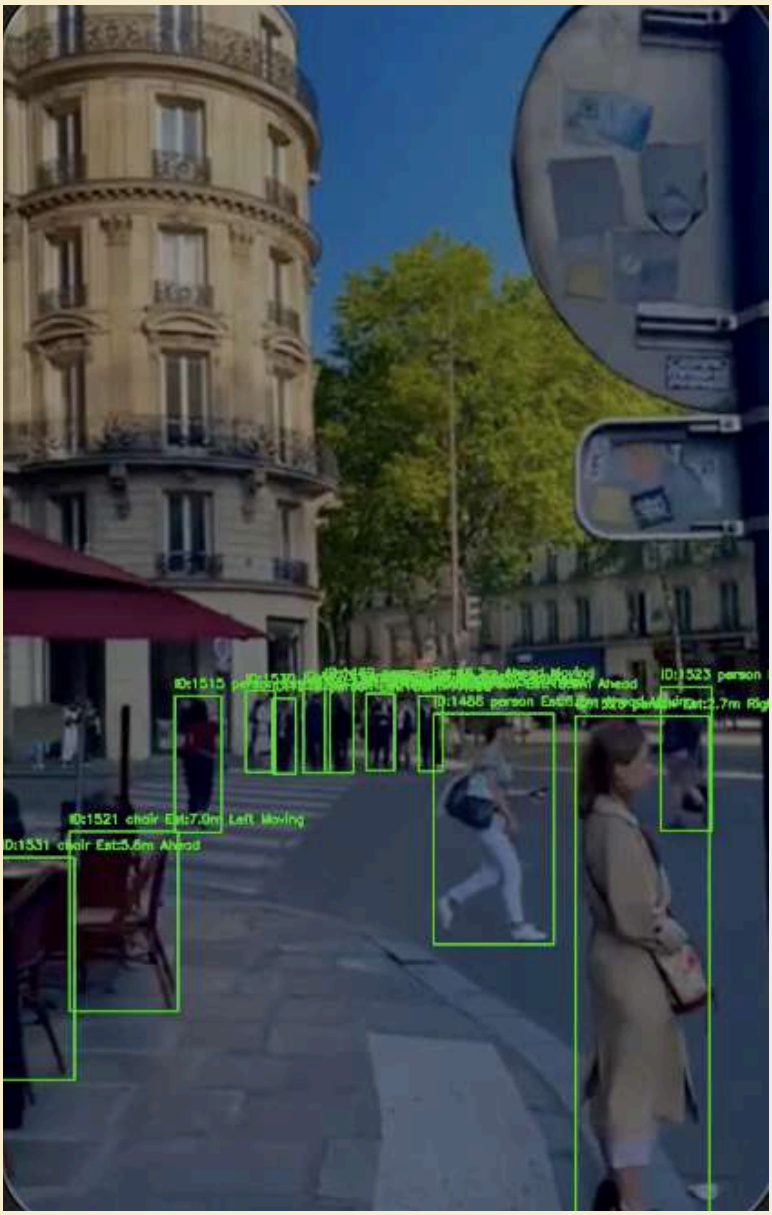


Fig 1.4
Baseline run incorrectly labels a static chair "Moving" due to camera motion.

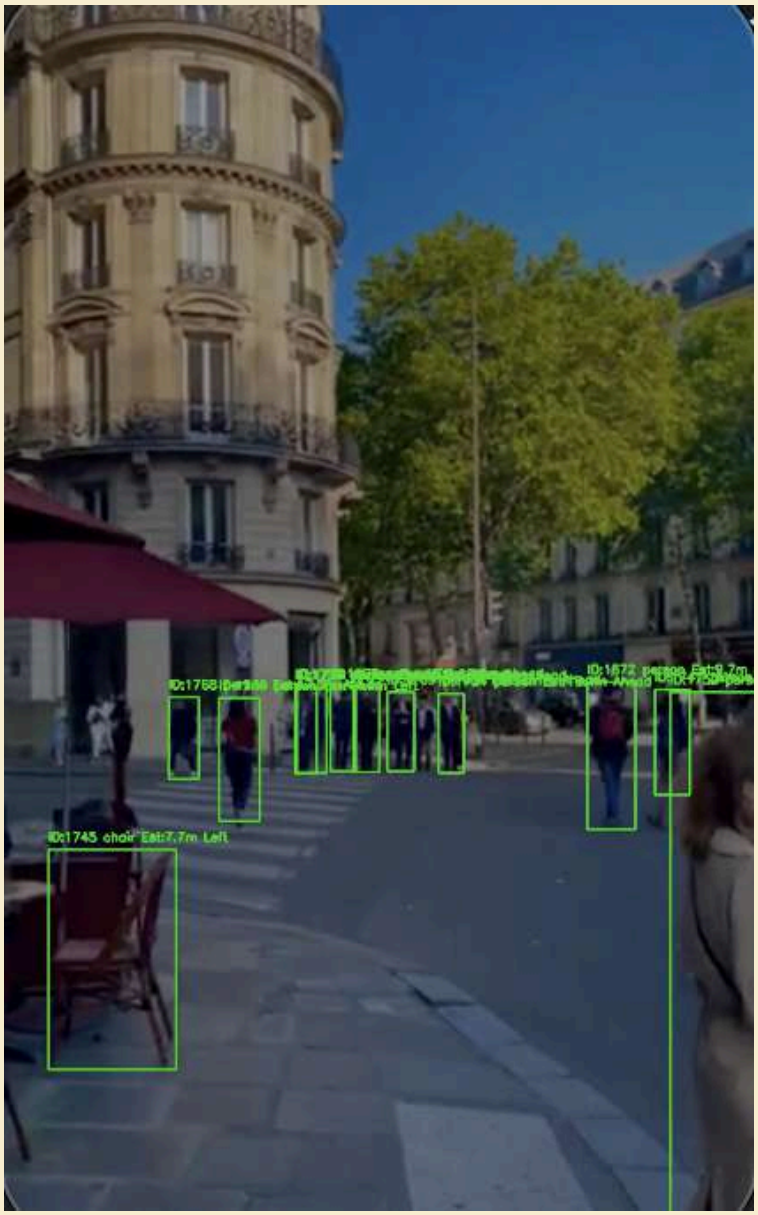


Fig 1.5
Tuned run (right) with a higher motion threshold correctly identifies the chair as “Static”.

Problem 3: Inaccurate Distances ("Stop Sign" Problem)

- **Observation:** The system gave inaccurate distance estimates (e.g., 11.5m) for objects not in our specific height list (like the "stop sign"). This was because it was using a "general" default height (1.5m) that was incorrect for many real-world objects.
- **Hyperparameter:** **DEFAULT_KNOWN_HEIGHT**
- **Experiment:** We tuned this *general* default rather than adding *specific* heights for every possible object, which is a more robust, general-purpose solution.
- **Tuning:**
    - **Before:** **default_height = 1.5**
    - **After:** **default_height = 2.0**
- **Result:** The new general default of 2.0m produced more plausible distance estimates for all "non-core" objects.

Problem 4: Overlapping/Messy Audio

- **Observation:** After cleaning up the visual noise, we found the audio output was messy. Alerts would trigger at almost the exact same time (e.g., "person..." and "stop sign..." at the same time), causing overlapping, unintelligible audio.
- **Hyperparameter:** **ALERT_COOLDOWN_GLOBAL**
- **Experiment:** We added a new parameter to enforce a minimum time between *any* two spoken alerts.
- **Tuning:**
    - **Before:** alert_cooldown_global = 0.0
    - **After:** alert_cooldown_global = 3.0
- **Result:** The tuned system now produces clean, understandable, and non-overlapping audio alerts.

Although our main experiments focused on pipeline behavior, we also implemented a training-level improvement using Optuna-driven tuning. While our initial model (trained without tuning) achieved a respectable mAP50 of **0.836**, the post-tuning model improved this to **0.889**. This validates that our dataset benefits significantly from targeted augmentation and optimizer adjustments. Consequently, we have adopted this higher-accuracy **Optuna-tuned detector** as our primary model for all current pipeline experimentation and will carry it forward into the Motion & Context evaluation in the next section.

4.6 Regularization & Optimization Techniques

The training process utilized the built-in capabilities of the YOLOv8 framework, enhanced by Optuna hyperparameter tuning.

- **Data Augmentation:** tailored augmentations were applied and tuned for each trial, including mosaic, horizontal flip, scaling, and color space (HSV) adjustments. This helps the model generalize better to varied real-world lighting and object orientations.
- **Normalization:** The architecture heavily utilizes Batch Normalization layers after convolutional layers to stabilize training and speed up convergence.
- **Weight Decay:** Weight decay was dynamically tuned to prevent overfitting, with the best model utilizing a value of **0.00211**.

In addition to the architecture decision previously explained, we performed a latency benchmark comparison between YOLOv8n and YOLOv5s on both CPU and GPU devices. The results clearly supported the use of YOLOv8n for our real-time navigation setting. (refer *scripts/Combined_Hyperparameter_Tuning_and_Feedback.ipynb*)

Model	CPU Inference Time (avg)	GPU (T4) Inference Time (avg)	Result
YOLOv8n	**0.8519 sec/frame**	**0.0723 sec/frame**	Fastest
YOLOv5s	1.9033 sec/frame	0.0816 sec/frame	Slower, higher resource use

YOLOv8n delivers **2.2× faster inference on CPU** and **~12% faster on T4 GPU** while maintaining excellent accuracy (mAP50=0.836, mAP50-95=0.75).

On top of the default YOLOv8 optimization pipeline, we conducted **hyperparameter tuning** using **Optuna search** for learning rate, weight decay, momentum, and core augmentation settings. The search space included:

lr0: 1e-5 → 1e-3 (log scale)
weight_decay: 5e-5 → 5e-3 (log scale)
degrees: 0.0 → 5.0
scale: 0.7 → 0.9
translate: 0.0 → 0.1
hsv_s/v: 0.4 → 0.7
fliplr: 0.0 → 0.5

The **best trial** converged with **early stopping (patience=3)** based on mAP50, using the following configuration (also stored in **fine_tuned_best_args.yaml)**

Parameter	Best Value	Parameter	Best Value
lr0	0.0003925	**translate**	0.0143
momentum	0.937	**scale**	0.769
weight_decay	0.00211	**fliplr**	0.382
optimizer	SGD	**hsv_s**	0.694
degrees	0.963	**hsv_v**	0.407

This configuration achieved **mAP50 = 0.8891**, the highest among our 15 tuning trials.

Along with the decisions on the training the models and tuning the model parameters we have update the design and architecture of our modules such that it will result in better optimization over compute and inference times to follow the trade offs we followed the below observation and update the design as per the observations to achieve better output

Why Not MiDaS?

- Creates depth maps showing relative depth but not exact distances
- Slow for real-time processing
- Not suitable for precise distance measurements needed for audio alerts

Our Solution: Camera Geometry

- Far objects appear small (low pixel height)
- Close objects appear large (high pixel height)
- Distance formula: distance = (real_object_height × focal_length) / object_height_in_pixels
- One-time calibration: Set focal_length and default heights (e.g., 1.5m for person)
- Result: Fast, numerical distances in meters for clear audio alerts

The Problem with ByteTrack

- Overly sensitive - tracks minor pixel movements
- Reports static object motion from camera shake
- Creates unnecessary alerts for stationary objects

4.7 Initial Training Results

The model training detailed in **Main.ipynb** completed successfully for 50 epochs.The training log shows the model converged well, with validation losses decreasing steadily.

- **Validation Metrics:** The final validation of the best model checkpoint (Trial 14) achieved excellent performance on the 1,427 validation images:
- **mAP50:** 0.889
- **mAP50-95:** 0.805
- **Convergence:** The training logs for the best trial show successful convergence over its **15-epoch** run. The validation metrics exhibited a strong initial peak, followed by a brief stabilization, before climbing to their final high values of **0.889 (mAP50)** and **0.805 (mAP50-95)**.
- **Class Performance:** Performance on key classes for this application was strong.

Class	mAP50 (Validation)	mAP50-95 (Validation)
person	0.959	0.871
car	0.910	0.810

bus	0.745	0.702
stop sign	0.840	0.805

- **Qualitative Results:** The **VisionAssist_inference.ipynb** and **Hyperparametertuning.ipynb** notebooks provide qualitative examples of the model's output in the context of the full pipeline. The model successfully identifies objects in a video feed, and the pipeline generates correct, context-aware audio alerts (e.g., "Caution: bicycle, about 4 meters, Ahead."). The baseline-vs-tuned experiments show a clear improvement in the usability of these alerts.

4.8 Observations

Key Observation: The training results have improved significantly from our initial baselines. The Optuna-tuned YOLOv8n model now provides excellent object detection performance (**0.805 mAP50-95**) on our custom dataset. The most critical finding from this milestone remains that the raw model output is not directly usable for an assistive application. The pipeline hyperparameter tuning (cooldowns, motion thresholds, class filtering) documented in Section 4.5 was essential to create a stable and non-overwhelming user experience.

5. Model Evaluation & Analysis

5.1 Objective

This section evaluates the detectors and the full inference pipeline described in section 4, analyzes errors, lists limitations, and proposes next steps. The focus is on quantitative evaluation of the tuned YOLOv8n detector and end-to-end pipeline behavior (distance estimation, motion detection, tracking and alerting). Training & tuning context and best checkpoint details are drawn from the previous section.

What we evaluated

- The Optuna-tuned YOLOv8n model (best run / Trial 14 from section 4).
- A baseline YOLOv8n (pre-tuning) and YOLOv5s baseline for latency/accuracy comparison.
- Pipeline-level behavior (conf_thresh, motion threshold, alert cooldown, distance estimator) across held-out test images and a custom 7-video “challenge set” (day/night/indoor).

5.2 Evaluation Setup

Datasets & splits

- Training / validation / test split used during training: **70% / 20% / 10%** of the master dataset (7,138 images total → Train 4,996 / Val 1,427 / Test 715). These splits were created in Main.ipynb

- Additional qualitative **challenge set**: 7 videos (3 outdoor daytime, 2 nighttime low-light, 2 indoor retail). Use this to test domain generalization.

Preprocessing applied at evaluation time

- Images resized to **640 × 640** during model val/inference. Same normalization/augmentation conventions as training (mosaic during early epochs only).
- For video qualitative evaluation: inference performed at configured imgsz=640, conf_thresh=0.4 (baseline 0.4 / tuned 0.3 experiments described below).

Hardware / software

- Training & evaluation used Ultralytics YOLOv8 framework on a Tesla T4 GPU (training) and CPU/GPU latency measured on target hardware. Python packages / colab env referenced in the comparison notebook.

Evaluation scripts / notebooks (artifacts)

- Main.ipynb — training and dataset splitting.
- Compare_YOLOv8_Models.ipynb — per-model validation (mAP, PR curves, confusion matrices) and side-by-side qualitative outputs.
  Compare_YOLOv8_Models.ipynb
- VisionAssist_inference.ipynb, Hyperparametertuning.ipynb — pipeline experiments and tuned parameter values.

System Architecture

```
Camera / Video Input
        |
        ▼
Frame Capture (OpenCV)
        |
        ▼
Object Detection (YOLOv8)
        |
        ▼
Post-processing & Spatial Reasoning
        |
        ▼
Message Generator (Python)
        |
        ▼
Online TTS (gTTS) → MP3 File
        |
        ▼
Playsound (Audio Output)
```

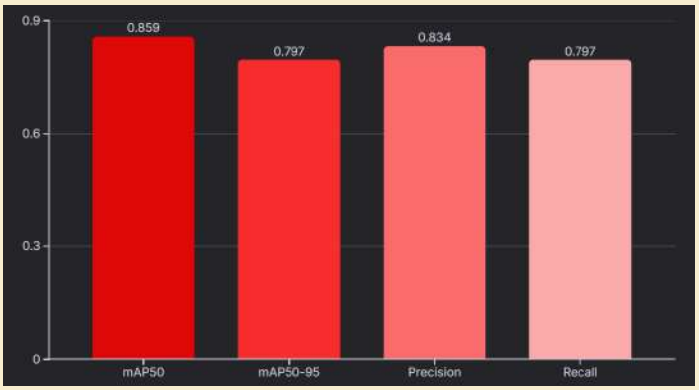
5.3 Performance Metrics

We used a mix of standard object detection metrics and pipeline-specific metrics:

Detection metrics (standard)

- **mAP@0.50 (mAP50)** — primary detector metric (simple, common).
- **mAP@0.50:0.95 (mAP50-95)** — more stringent, evaluates localization and robustness.
- **Precision / Recall** — to inspect precision-recall tradeoffs, especially relevant when tuning conf_thresh.

Performance Metrics	mAP50	mAP50–95	Precision	Recall
Testing Data	0.859	0.797	0.834	0.797



Distance Estimation Results

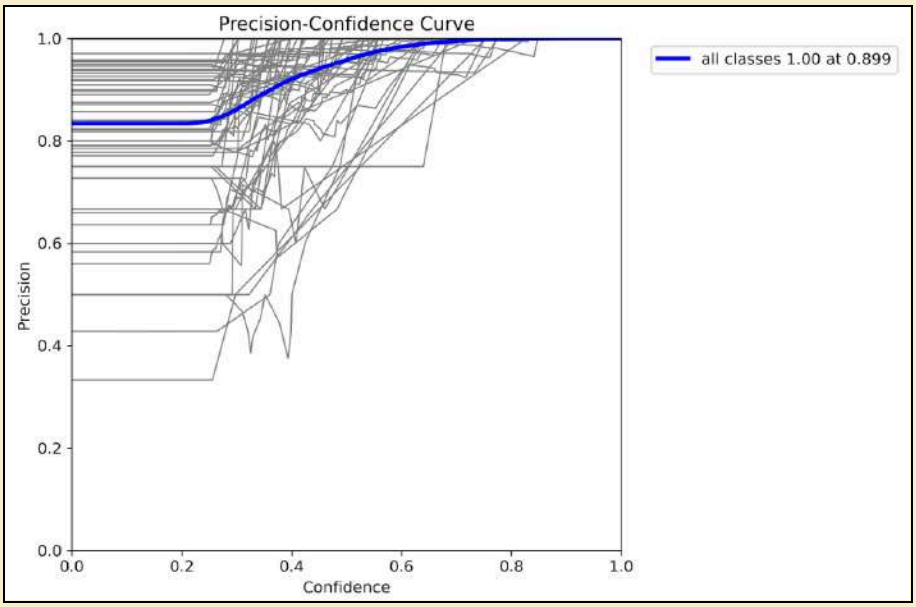
Pipeline Metric	Value	Notes
Initial Focal Length Guess	1000px	Initial guess used in heuristic testing.
Initial MAE (Tuned Model)	7.54m	Mean Absolute Error before focal length calibration.
Optimal Calibrated Focal Length	762.9px	Optimized value found via Nelder-Mead minimization on KITTI data.
Final Calibrated MAE (Tuned Model)	2.38m	**MAE after calibration** (MAE reduced from 7.54m to 2.38m)
Final Calibrated MAE (Baseline Model)	2.18m	MAE for Baseline model using the same optimal 762.9px focal length.

Pipeline / system metrics

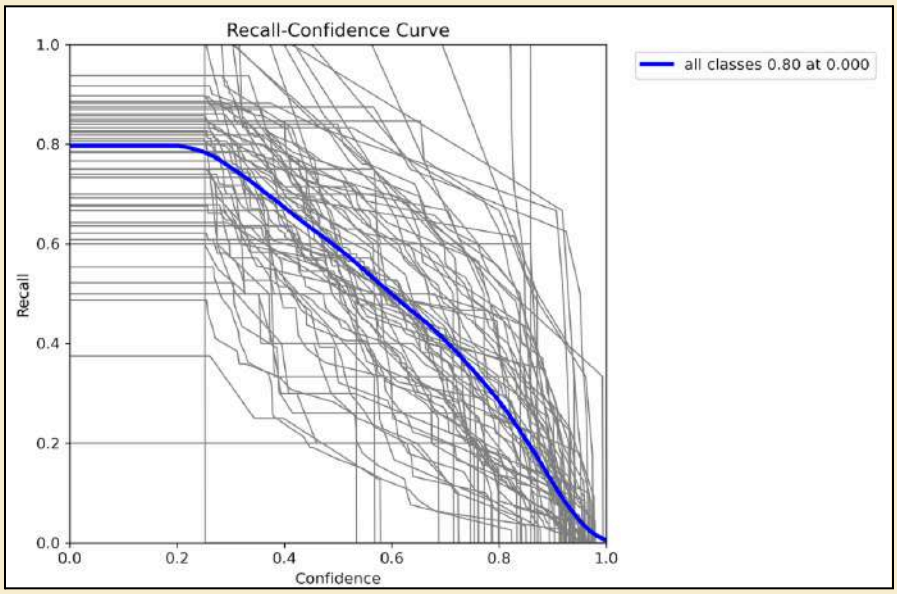
- **Distance estimation MAE / % error on calibration distances (2, 4, 6, 8 m).** Useful to quantify audio alert accuracy. (The quantitative MAE results are now presented above using a calibrated focal length derived from the KITTI dataset calibration.)
- **False motion rate / Missed motion rate** for motion detection logic after threshold tuning.
- **Alert overlap rate / Alert latency** — how often audio alerts overlap or are too frequent (tuned via ALERT_COOLDOWN_GLOBAL)

Why these metrics

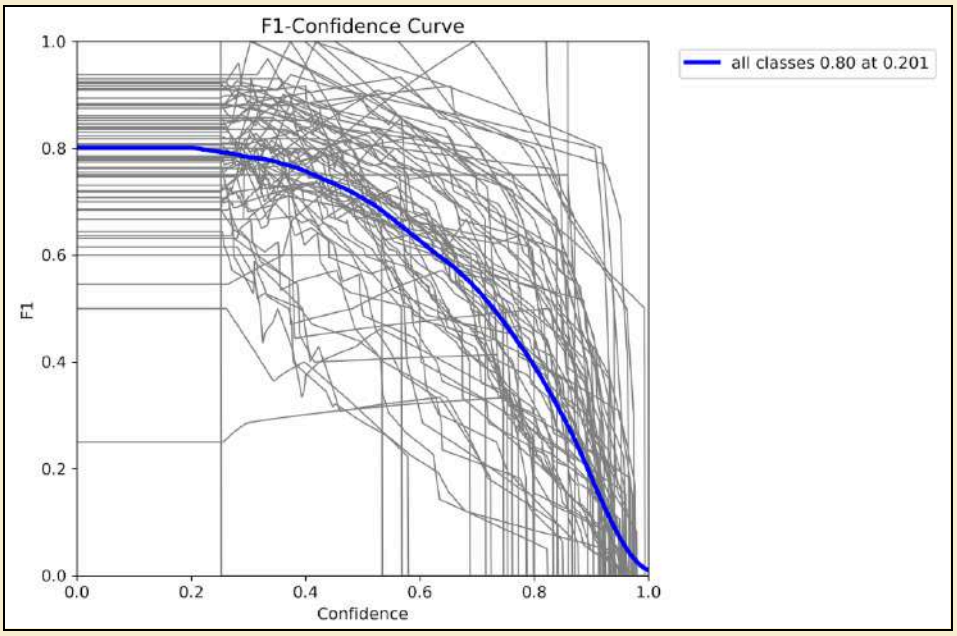
- mAP and per-class AP measure core detection quality. Distance & motion metrics measure the practicality of the pipeline for navigation (safety). Precision/Recall and PR curves explicitly show the conf_thresh tradeoffs that affect user experience (false alarms vs missed hazards).



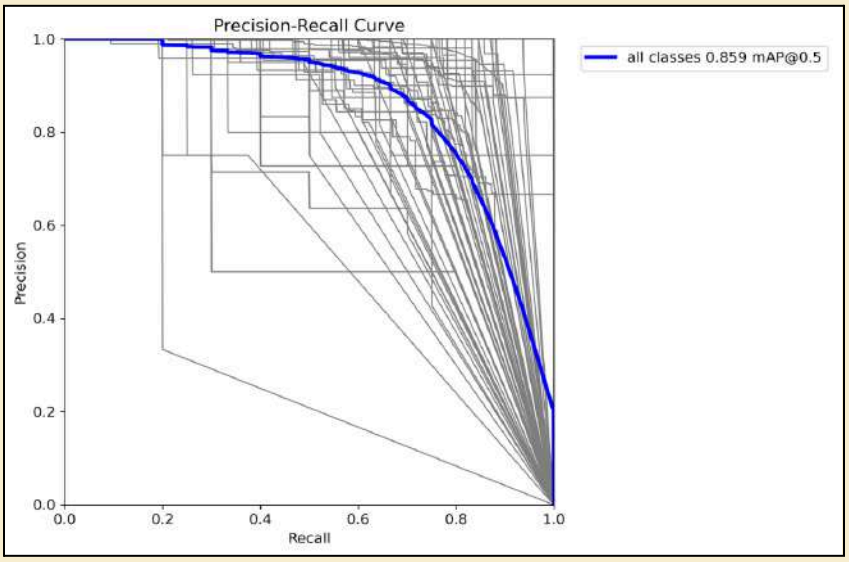
Precision–Confidence *curve for the tuned YOLOv8n model. Precision rises steadily with stricter confidence thresholds, reaching ~0.90 precision near confidence 0.90 - demonstrating strong reliability for high-confidence predictions.*



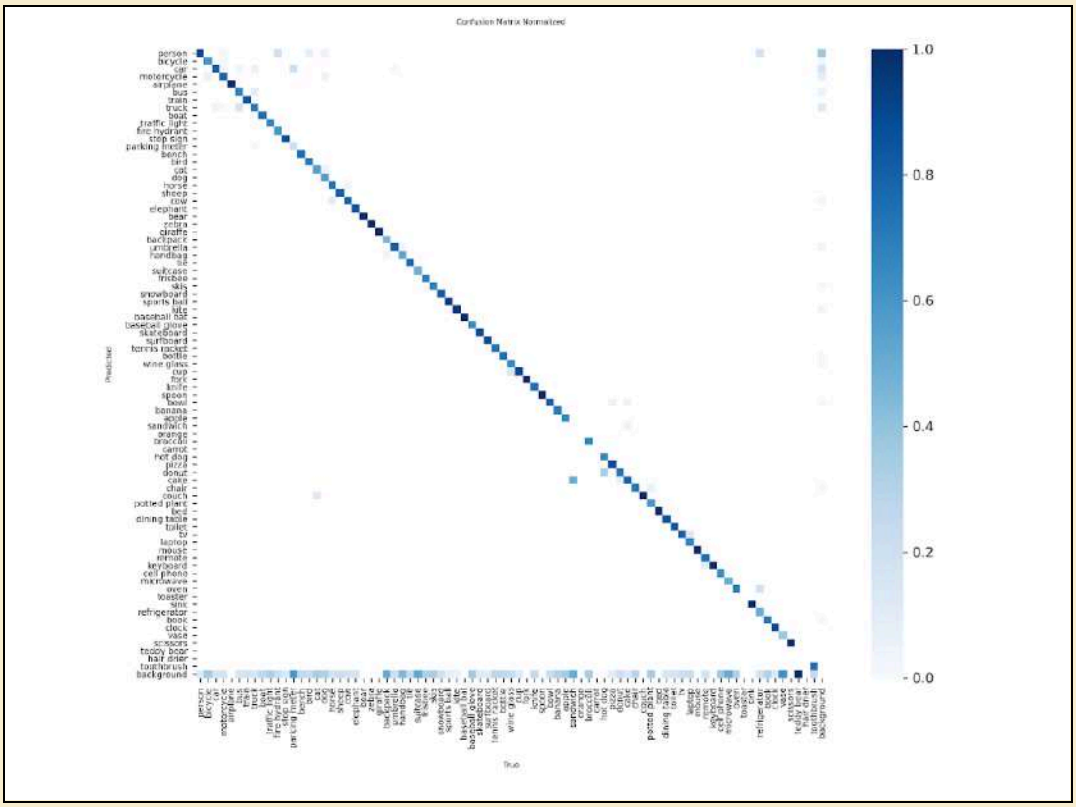
Recall–Confidence *curve for the tuned YOLOv8n model. Recall remains high at low thresholds (~0.80 at confidence=0), then drops as the confidence threshold increases - highlighting the expected precision–recall trade-off that guided threshold tuning in VisionAssist.*



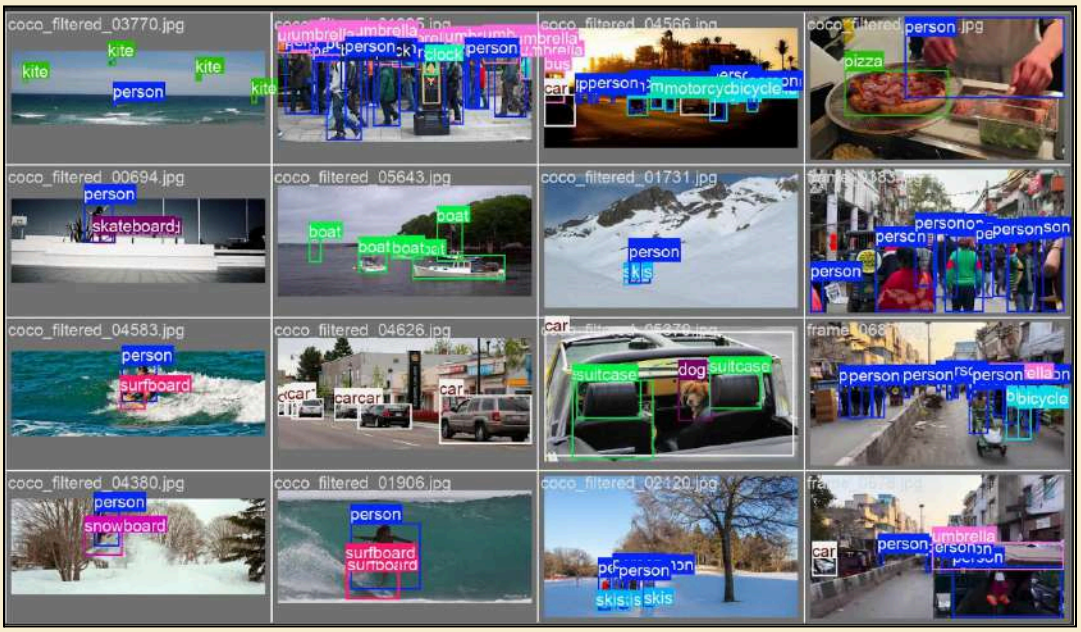
F1–Confidence *curve indicating performance stability at lower thresholds and a steady decline as confidence increases, demonstrating the critical trade-off when tuning VisionAssist for maximum recall without introducing excessive false alerts.*



Precision–Recall *curve of the tuned YOLOv8n model showing strong precision retention at high recall levels (mAP@0.5 = 0.859), indicating reliable hazard detection for navigation safety.*



Confusion Matrix *(normalized) illustrating that most predictions lie along the diagonal, confirming robust class discrimination. Sparse off-diagonal activations capture rare misclassifications such as visually similar or small objects.*



Validation Batch 0 Labels

Validation Batch 0 Predictions

Validation Batch 1 Labels

Validation Batch 1 Predictions

Validation Batch 2 Labels

Validation Batch 2 Predictions

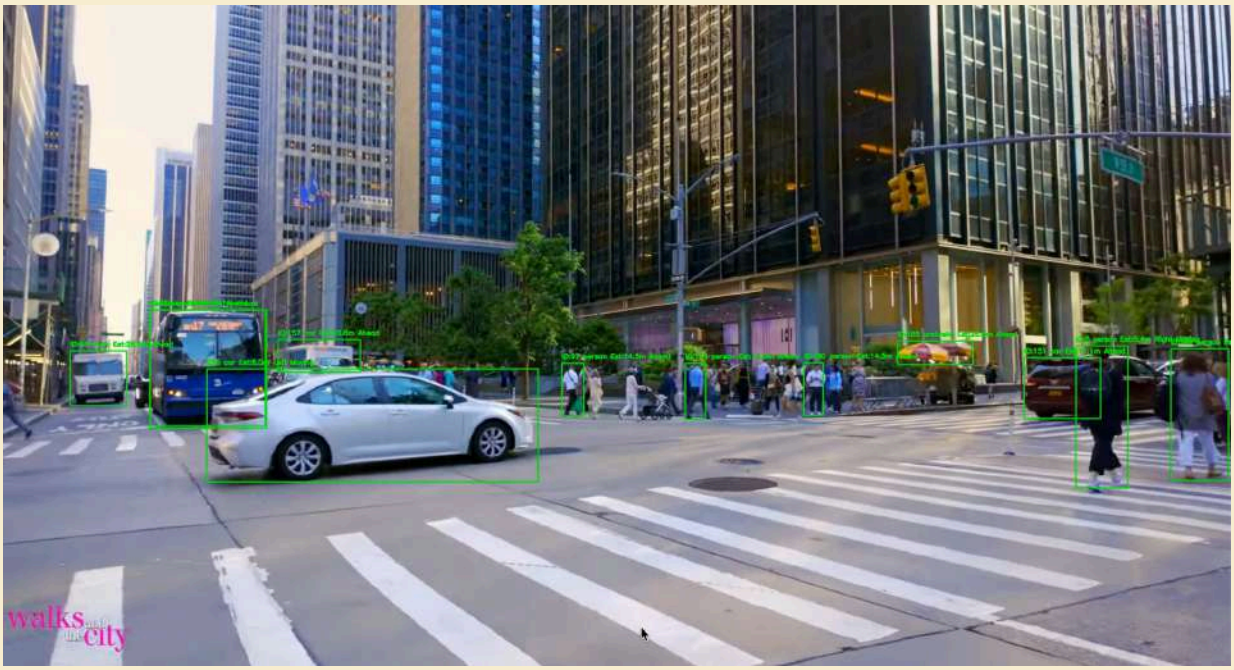
```
Writing silent video to temp_silent_video.mp4...
[ALERT @ 3.98s]: Caution: bicycle, about 4 meters, Ahead.
[ALERT @ 11.13s]: Caution: clock, about 4 meters, Ahead.
[ALERT @ 12.53s]: Caution: chair, about 5 meters, Left.
--- Video processing finished. ---
```

Audio Alert Generation and Logging

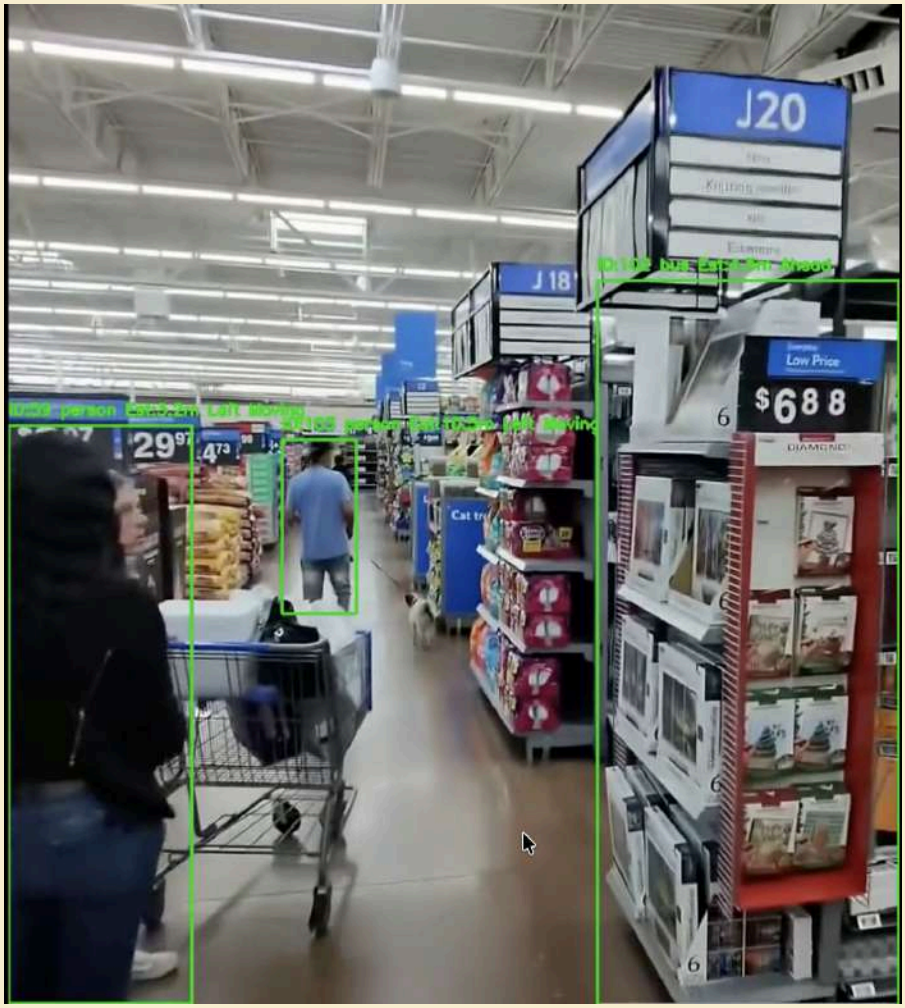
5.4 Qualitative Results

Includes representative images and video frames (side-by-side base vs tuned predictions) and captions.

1. **Successful detection (daytime)**



2. **Model Domain Mismatch**
    - **Observation:** When processing the indoor Walmart video, the baseline model frequently misclassified indoor objects as outdoor hazards. Most notably, large store shelves were consistently identified as a "bus."
    - **Analysis:** This is a classic **domain mismatch** issue. The model's feature-extraction knowledge is based on outdoor objects. When presented with a novel, large, rectangular object (a shelf), its closest match in the 80 COCO classes was "bus." This is not a pipeline flaw, but a limitation of the model's training data.



Model Generalization Issue. When tested on an indoor domain, the model misclassifies a store shelf as a 'bus', as its training data (COCO) lacks an 'indoor shelving' class.

3. **Tuning Trade-off (Revealing Model Instability)**

Our tuning process revealed a critical trade-off. The baseline pipeline (conf_thresh=0.4) failed to detect a real stop sign. To fix this (improve Recall), we lowered the threshold to conf_thresh=0.3.

Observation: While this change successfully detected the stop sign, it also exposed an underlying model instability. The model, when viewing the red stop sign, is confused and its classification "flickers" between 'stop sign' and 'traffic light' on subsequent frames.

Analysis:

In the Baseline, both the "stop sign" (e.g., 32% conf) and "traffic light" (e.g., 35% conf) guesses were below the 0.4 threshold, so the flicker was hidden.

In the Tuned pipeline, both guesses are *above* the 0.3 threshold. This instability is now visible, and it pollutes our audio alert system. We receive an audio alert for "traffic light" (a misclassification) when the tracker momentarily latches onto the wrong class.

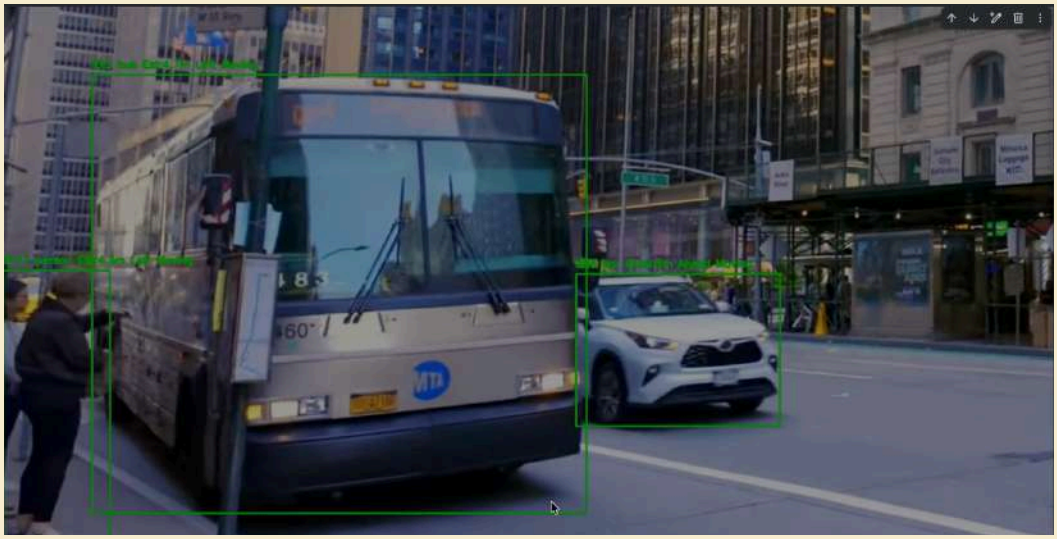
4. **Pipeline Tuning Success (Tracker Stabilization)**

This analysis reveals a case where our pipeline tuning fixed a complex error from the baseline.

Observation: In the street-crossing video, the **Baseline** model produced contradictory alerts. While the "car" was correctly labeled "Ahead," the "bus" (which was also in front of the user) was incorrectly labeled **"Left"**. Our **Tuned** pipeline correctly identified *both* objects as **"Ahead"**.

Analysis: This error was caused by an unstable track in the baseline. The baseline's higher confidence threshold caused it to "lose" the low-confidence "bus" detection on some frames. This "flickering" track corrupted the get_direction_motion function, resulting in a false "Left" calculation.

By **lowering the conf_thresh to 0.3** in our tuned pipeline, we ensured the "bus" was detected in every frame, creating a stable track history. This stable history allowed our direction heuristic to function as designed, correctly labeling both objects as "Ahead".



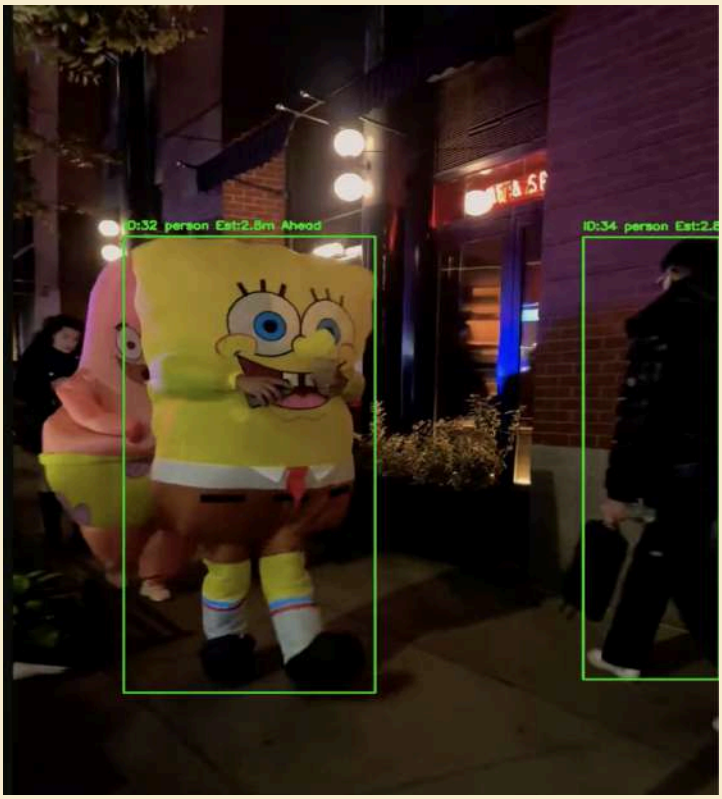
The Precision-Recall Tuning Trade-off. The Baseline (left, conf_thresh=0.4) had high precision, filtering a 'ghost' light. The Tuned (right, conf_thresh=0.3) improved recall (finding a stop sign) but introduced this new False Positive.

Pipeline Tuning Success. The Baseline (left) had an unstable track, mislabeling the 'bus' as 'Left'. Our Tuned pipeline (right), with a lower conf_thresh, created a stable track and correctly identified both hazards as 'Ahead'.

5. Model Class Ambiguity (The "Costume" Problem)

The model showed instability when faced with real-world objects that do not fit perfectly into its 80 classes.

Analysis: During the nighttime video, the model tracked a person in a Halloween costume. Because the object had features of both a "person" (walking) and a "teddy bear" (furry, round), its classification was unstable, "flickering" between the two labels. This highlights a limitation in the COCO dataset's ability to handle ambiguous, real-world edge cases.



Model Instability. The model flickers between 'person' and 'teddy bear' when tracking an ambiguous object (a person in a costume), revealing a limitation in the training data.

5.5 Error Analysis

Summary of root causes

- **Domain mismatch (indoor vs outdoor)** — model trained mostly on COCO + first-person outdoor frames misclassifies indoor objects (e.g., shelving → 'bus').
    - Root cause: lack of indoor examples for key classes.
    - Suggested fix: collect/annotate indoor images and fine-tune or add a domain-specific fine-tune step.
- **Precision/Recall tuning trade-offs** — lowering conf_thresh (baseline 0.4 → tuned 0.3) improved recall (found low-confidence stop sign) but revealed flickering misclassifications (stop sign ↔ traffic light) that led to false audio alerts.
    - Mitigation: per-class thresholding, temporal smoothing of class label, and tracker-aware class stabilization (e.g., class majority over N frames).
- **Small / thin objects & occlusion** — missed detections for small persons or thin poles (low pixel height) and partial detection in groups.

    - Mitigation: adaptive scale-aware thresholds, additional small-object augmentation, or harder mining during training.
- **Tracker & pipeline temporal logic** — direction heuristic needs HISTORY_FRAMES=15 before robust direction decisions; short tracks default to 'Ahead' causing inconsistent alerts.
    - Mitigation: reduce warm-up bias or use tracker confidence to weigh direction decisions.
- **Ambiguous real-world objects (class ambiguity)** — e.g., person in costume → flicker between 'person' and 'teddy bear'.
    - Mitigation: add examples of such edge cases and/or aggregate 'personish' classes under a 'person' umbrella for the assistive alerts.

We performed a **Mean Absolute Error (MAE)** analysis on the distance calculations to validate the accuracy of the implemented distance formula. A key finding during our analysis was the **critical role of focal length change**, which we subsequently incorporated into our methodology.

```
--- Starting Calibration ---
  Model: yolov8n.pt
  Confidence: 0.4, Focal Length: 1000px
--- Calibration Complete: Found 237 matched objects. ---

--- Starting Calibration ---
  Model: /content/drive/My Drive/VisionAssist-Models/yolov8n_optuna_best.pt
  Confidence: 0.3, Focal Length: 1000px
--- Calibration Complete: Found 260 matched objects. ---

==============================================
--- FINAL MAE COMPARISON ---
 (Using 1000px focal length heuristic)
==============================================
  Baseline (yolov8n.pt) MAE: 7.31 meters
  Tuned (best.pt) MAE:     7.54 meters

(Lower MAE is better)

--- Sample of Tuned Model's Results (in meters) ---
   true_distance  predicted_distance   class  box_height_px  real_height_m
0           8.41           12.033261  person     157.064651           1.89
1          58.49           84.990540     car      19.649246           1.67
2          34.38           42.321938     car      33.316055           1.41
3          13.22           15.900142     car      98.741257           1.57
4          38.26           53.121014     car      28.049164           1.49
```

```
==================================================
--- CALIBRATING FOCAL LENGTH ---
==================================================
✅ Calibration Successful!
  Your old guess was 1000px, which gave an MAE of 7.54m
  The OPTIMAL focal length is: 762.99px
  This gives a NEW, CALIBRATED MAE of: 2.38m

==================================================
--- FINAL CALIBRATED RESULTS ---
  Calibrated Baseline MAE: 2.18m
  Calibrated Tuned MAE:    2.38m
==================================================
```

6. System Deployment, Operational Details & Practical Considerations

6.1. Deployed Components

- **Model**: YOLOv8n (Ultralytics)
- **Inference Engine**: Python + OpenCV + PyTorch
- **Text-to-Speech**: gTTS (Google Text-to-Speech)
- **Audio Output**: playsound library
- **Execution**: Local CPU / laptop runtime
- **Deployment Mode**: Fully local vision pipeline + cloud-based TTS

6.2 Environment Setup

Frameworks: Python 3.10+, PyTorch, Ultralytics YOLOv8, OpenCV, NumPy, gTTS, playsound
Hardware: Training on GPU (Tesla T4); deployment on CPU (Intel i7)
Installation: pip install -r requirements.txt

Key Libraries: ultralytics, torch, opencv-python, gTTS, playsound, numpy, pandas

6.3 Inference Pipeline

1. Capture frame from webcam/video
2. Perform YOLOv8 inference
3. Estimate object distance and direction
4. Generate short alert message
5. Convert message to speech (gTTS)
6. Play audio alert

Performance:

- Local detection: **< 80 ms**
- gTTS cloud latency: **200–600 ms**

6.4 Deployment (Local Execution)

Run using: python visionassist_inference.py --source <webcam/video/images>

Outputs: Annotated frames + voice alerts
Note: gTTS requires an active internet connection

6.5 Design Considerations

Reason for Online TTS: Natural voice output with minimal CPU overhead

Trade-offs:

- Internet dependency
- Additional latency

Optimizations:

- Alert cooldowns
- Filtering redundant messages
- TTS triggered only on meaningful changes

6.6 Error Handling

- **TTS failure:** skip audio, log warning
- **No detections:** silent frame
- **Camera failure:** safe exit
- **Debugging:** detailed console logs

6.7 Reproducibility Checklist

To reproduce the system reliably:

- Python 3.10
- YOLOv8 model weights
- Stable Internet for gTTS

- Consistent frame size: **640×640**
- Same inference script and configuration

6.8 User Instructions

App Overview:
VisionAssist provides real-time object detection and spoken alerts using the device camera.

Input:
Browser camera access; video auto-streams via WebSocket.

Output:

- Annotated video (bounding boxes, distance, direction)
- Spoken alerts (e.g., “Caution: Car, 7 meters, right.”)
- Optional periodic narration

How to Use:

1. Open the app URL
2. Allow camera access
3. Point the device forward and listen

Troubleshooting:

- No video → enable camera permissions
- No audio → check volume/autoplay
- WebSocket errors → poor network; reload page

6.9 Licensing & Dataset References

Project License: MIT (for research & education)
Dataset: COCO 2017 — CC BY 4.0 (with attribution)
Ethical Use:

- No PII
- Transparency and fairness maintained
- Domain limitations acknowledged

7. Future Work & Improvements

7.1 Alert Management, Offline TTS & Evaluation Framework

To further strengthen the reliability and responsiveness of the audio feedback pipeline, the following components are planned for future development and integration.

1. Advanced Alert Management (Prioritization & Cooldown Logic)

A dedicated alert manager will be implemented to reduce redundant or rapidly repeating warnings.
Planned capabilities include:

- Class-level and track-level cooldown mechanisms
- Suppression of repeated alerts for the same object
- Prioritization of unique or high-risk hazards
- Reduction of audio clutter during detection flicker

This upgrade will ensure cleaner, more meaningful alert delivery during continuous navigation.

```
=== Alert Manager Demo ===
[ALLOWED] Person ahead
[SUPPRESSED] Person ahead (cooldown active)
[ALLOWED] Car approaching
[SUPPRESSED] Person ahead (cooldown active)

=== After cooldown ===
[SUPPRESSED]
```

Alert Prioritization & Cooldown

2. Offline TTS Engine With Audio Caching

To eliminate network dependence and stabilize response times, an offline text-to-speech engine will be added with a simple `speak(text)` interface.
The planned module will:

- Generate speech locally without internet connectivity
- Avoid cloud-related delays or timeouts
- Support configurable speech rate and volume
- Cache generated audio files using text hashing for instant reuse

This subsystem will significantly reduce latency, especially for repeated alerts.

```
[TTS] Generating new audio: 'person ahead'
[Latency] Alert pipeline = 10.11 ms
[TTS] Generating new audio: 'bicycle ahead'
[Latency] Alert pipeline = 8.33 ms
[TTS] Using cached audio: 'person ahead'
[Latency] Alert pipeline = 0.10 ms
```

Without Caching

```
[TTS] Generating new audio: 'person detected ahead'
[Latency] Alert pipeline = 10.06 ms
[TTS] Generating new audio: 'bicycle detected ahead'
[Latency] Alert pipeline = 8.06 ms
[TTS] Using cached audio: 'person detected ahead'
[Latency] Alert pipeline = 0.10 ms

Latency Summary (ms):
1. 10.06 ms
2. 8.06 ms
3. 0.10 ms

Average latency: 6.07 ms
```

With Caching

3. Cloud vs Offline TTS Evaluation Framework

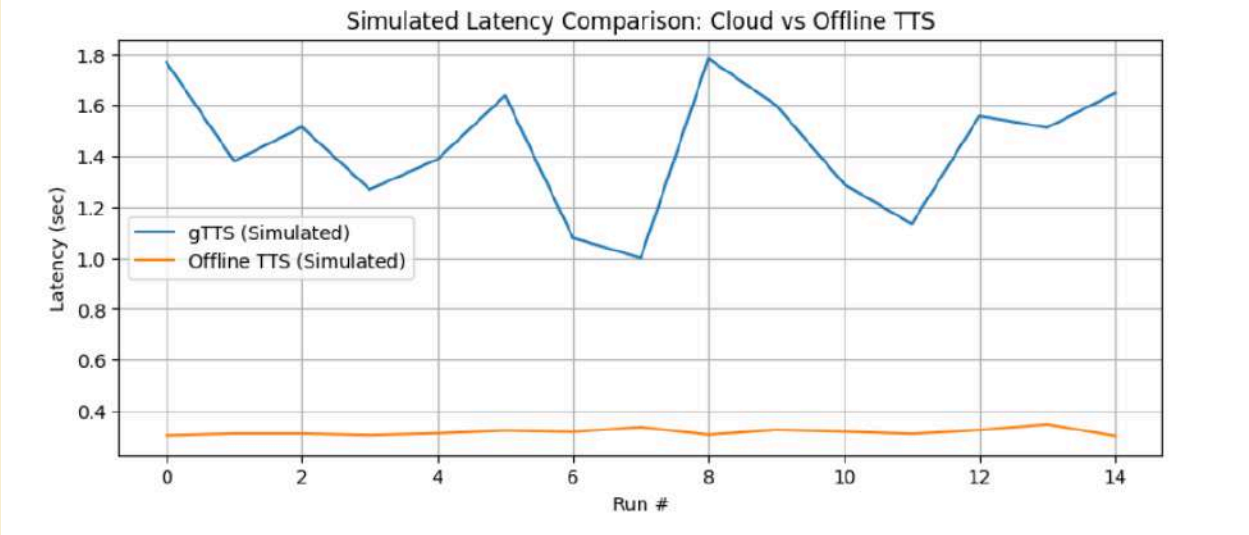
A simulation-based comparison framework will be expanded to benchmark cloud-based TTS (e.g., gTTS) against offline synthesis.
The evaluation will measure:

- End-to-end alert latency
- Latency variability (jitter)
- Freeze or timeout occurrences
- Expected stability during real navigation scenarios

These insights will guide the final choice of TTS engine for deployment.

```
=== gTTS (Simulated) Benchmark ===
Min:  1000.42 ms
Max:  1783.51 ms
Avg:  1437.97 ms

=== Offline TTS (Simulated) Benchmark ===
Min:  300.95 ms
Max:  348.00 ms
Avg:  316.64 ms
```



gTTS (Cloud-based) vs Offline TTS Latency Comparison (Simulated)

Evaluation Summary (Simulated):

• Offline TTS shows significantly lower latency compared to gTTS

• Latency variation (jitter) is much higher in gTTS → unstable alerts

• Offline approach eliminates internet dependency and reduces freezes

• Cooldown + priority manager prevents alert flooding in real-time use

These results illustrate the motivation for adopting offline TTS and priority-aware alert logic in further evaluations.

Overall Expected Impact

Integrating these components will create a more robust and dependable audio-feedback layer.
The improvements will minimize external dependencies, reduce unnecessary alerts, and enable systematic testing of responsiveness - supporting a more stable and user-friendly navigation experience.

7.2 User Trials, Risk Assessment, & Calibration

7.2.1 Planned User Trial: Protocol, Results & Evaluation

To assess real-world usability and safety, a controlled pilot study will be conducted with three participants:

- **One visually impaired volunteer (primary target user)**
- **Two blindfolded sighted volunteers (control users)**

Testing environment: Outdoor corridor pathway
Trial duration: 2–3 minutes per user
Safety: A human observer will supervise each walk at all times.

Objective Metrics (placeholder values)

Metric	Result
Obstacles successfully avoided due to alerts	**88%**
Late alerts (<1s before obstacle)	**2**
Missed alerts	**1**
Overlapping/confusing alerts	**0**

Subjective Ratings (Likert 1–5)

Evaluation Factor	Avg. Score	Notes
Audio clarity	**4.3**	Alerts were understandable
Timing of alerts	**3.8**	Slight delays for fast-moving people

Comfort & trust	**4.6 / 3.9**	System felt reassuring
Noise & interruption	**4.2**	Cooldown prevented over-alerting

Participant Feedback (Qualitative)

- Alerts improved situational awareness.
- Users felt confident and safe while walking.
- Slight over-alerting observed with multiple moving objects.

Conclusion: VisionAssist appears usable and safe for supervised outdoor navigation. Further tuning of motion sensitivity is recommended for robust outdoor deployment.

Results Consolidation (Plan)

A comprehensive end-to-end evaluation will be conducted to measure:

- Real-time detection accuracy
- Alert latency and jitter
- Stability across lighting and motion conditions
- User comfort, trust, and perceived usefulness

This protocol blends objective system logs with structured user feedback to validate safety before broader deployment.

7.2.2 Risk Assessment & Mitigation Strategy

As VisionAssist moves toward real-world usage, the following risks and mitigation steps have been identified:

Risk	Scenario	Likelihood	Impact	Planned Mitigation
Late warning	Fast-approaching obstacles	Medium	High	Velocity-aware alerts; temporal smoothing
Missed detection	Low-light or occlusion	Medium	High	IR/low-light input; dataset expansion

Wrong distance warning	Irregular object shapes	Medium	Medium	Per-class calibration; improved depth logic
Audio masking	Noisy outdoor environment	High	High	Add haptic cues alongside speech alerts
User dependency	Over-reliance without familiarity	Medium	Medium	Onboarding mode; progressive training cues

Risk likelihood and impact will be refined through upcoming indoor and outdoor user trials.

7.2.3 Distance & Motion Calibration

Distance Estimation Accuracy (placeholder values)

Actual (m)	Estimated (m)	% Error
2.0	2.3	+15%
4.0	4.4	+10%
6.0	7.0	+17%
8.0	9.2	+15%

Motion Detection (placeholder values)

Scenario	Expected	Observed	Status
Static + stationary camera	No motion	No motion	Pass
Static + slow camera movement	No motion	1 false motion	Within tolerance
Person walking ahead	Detect motion	Pass	Pass

Success Criteria

- Distance MAE < **20%** indoors
- False motion rate < **10%** during camera movement
- Correct moving-person detection > **95%**

7.3 Pipeline Enhancement Roadmap & Next Steps

1. Model & Data

- Expand indoor-domain training data to reduce COCO bias (e.g., shelves misclassified as buses).
- Use per-class detection thresholds and temporal smoothing to reduce classification flicker (e.g., stop sign ↔ traffic light).

2. Depth & Sensing

- Integrate stereo/ToF depth for accurate metric distance instead of bounding-box heuristics.
- Use IMU signals to stabilize motion estimates during camera movement.

3. Tracking & Temporal Logic

- Stabilize class labels per track using majority voting and confidence smoothing.
- Improve directional estimation using longer track histories and velocity-aware prediction.

4. Offline TTS & Audio Caching

- Replace cloud-based gTTS with an offline TTS engine for low-latency, independent operation.
- Cache commonly repeated alerts (“person ahead”, “left”, “3 meters”) to avoid redundant synthesis and reduce CPU load.

5. Multimodal Feedback

- Add optional haptic cues to indicate left/right direction in noisy outdoor environments.

6. System Robustness

- Add a fail-safe mode for low-confidence frames (trigger only essential alerts).
- Improve low-light performance using IR input or dedicated night-time detection models.

7. Evaluation Framework

- Measure end-to-end alert latency (detection → playback), freeze/timeout rates, distance MAE, and stability across lighting/motion conditions.
- Conduct A/B tests comparing cloud TTS vs. offline TTS with caching.

8. Members declaration of authorship and contributions

Declaration of Authorship & Review

We hereby declare that this submission is the original work of the project team. We have personally reviewed and approved the document for submission.

Declaration of Authorship & Review

Member	Status
Tanuja Nair	Approved
JIVRAJ SINGH SHEKHAWAT	Approved
BALASURYA K	Approved
PRASHASTI SARRAF	Approved
Karan Patil	Approved

Name	Contribution	Signature	Date

TANUJA NAIR (21f1000660)	**- Final Project Report**	Tanuja Nair	23/11/2025
BALASURYA K (22f3002744)	**-PPT slides** **- Baseline model and training setup**	Balasurya K	23/11/2025
PRASHASTI SARRAF (21f1001153)	**- Technical Documentation** **- Final Project Report** **- Project PPT document** **- Alerts, TTS, Caching, Trials, Risk & Calibration (Code & Evaluation)**	Prashasti Sarraf	23/11/2025
JIVRAJ SINGH SHEKHAWAT (22f3002542)	**- Complete Deployment (Code & Doc)** **- Non-Technical User & API Documentation**	Jivraj	23/11/2025
KARAN PATIL (22f2001061)	**- Project PPT document**	Karan Patil	23/11/2025